

Voluntario: Simulación del modelo de Ising con la dinámica de Kawasaki

Santiago Vallecillos Aranda.

En el presente informe se estudia el modelo de Ising con la dinámica de Kawasaki, que ha sido simulado mediante programas en C++ para un posterior análisis de datos mediante scripts de python. El objetivo ha sido estudiar la evolución de un sistema mediante esta dinámica con diferentes condiciones de contorno, así como cálculos sobre la energía del sistema, el calor específico, la temperatura crítica, la magnetización y la susceptibilidad magnética para magnetizaciones iniciales nula y no nula. Se pudo observar con claridad la influencia de las condiciones termodinámicas sobre las propiedades magnéticas de un sistema y se calcularon valores de temperatura crítica, susceptibilidad magnética y energía media. Asimismo, se comprobó que la densidad de partículas de cada espín del sistema es constante.

Índice

1. Introducción y fundamento teórico.
2. Material utilizado.
3. Código.
4. Resultados y discusión.
5. Conclusiones.

1. Introducción y fundamento teórico.

El modelo de Ising estudia unos materiales que sufren un cambio brusco de sus propiedades magnéticas con la temperatura, siendo paramagnéticos a temperaturas altas y ferromagnéticos a temperaturas bajas. El modelo se define en una red cuadrada bidimensional N^2 , donde en cada nudo de la red tiene un espín $s(i, j)$, $i, j = 1, 2, \dots, N$ que puede tomar los valores 1 o -1. La energía de interacción del sistema dada una configuración de espines es

$$E(C) = -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N s(i, j)(s(i+1, j) + s(i-1, j) + s(i, j+1) + s(i, j-1))$$

La dinámica de Kawasaki para este modelo, consiste en escoger un par de espines vecinos al azar e intercambiar sus valores con la probabilidad dictada por el algoritmo de Metrópolis. Este algoritmo consiste en generar configuraciones de espines a través de cadenas de Markov de eventos sucesivos $X_1, X_2, \dots, X_N$, que tiene como probabilidad:

$$P_N(X_1, X_2, \dots, X_N) = P_1(X_1)T(X_1 \rightarrow X_2)T(X_2 \rightarrow X_3) \dots T(X_{N-1} \rightarrow X_N)$$

donde $T(X \rightarrow Y)$ es la probabilidad de transición de ir del estado X al estado Y . De aquí se deduce la Ecuación Maestra, sea $g(X, t)$ la probabilidad de que en el paso t de la cadena de Markov nos encontremos en el estado X , la ecuación es:

$$g(X, t+1) = g(X, t) + \sum_{X' \neq X} [g(X', t)T(X' \rightarrow X) - g(X, t)T(X \rightarrow X')]$$

y para conseguir una solución estacionaria $g(X)$, pedimos la condición de balance detallado:

$$g(X')T(X' \rightarrow X) = g(X)T(X \rightarrow X')$$

La dinámica de Kawasaki conserva la magnetización inicial. Sin embargo, para una magnetización inicial nula, seguimos teniendo, por debajo de la temperatura crítica una transición de fase continua, obteniendo dominios de igual tamaño y de magnetización no nula, por tanto con signos contrarios para que la magnetización total siga siendo cero. Por el contrario, para una magnetización inicial no nula, esta dinámica da lugar a una transición de fase discontinua por debajo de cierta temperatura, dando lugar a la coexistencia de dominios de magnetización positiva y negativa de distintos tamaños.

La probabilidad de cambio de estado al utilizar el algoritmo de Metrópolis es $P_{C \rightarrow C'} = \min(1, e^{-\beta \Delta E})$, donde $\Delta E = E(C') - E(C)$ y $\beta = 1/T$ donde hemos considerado la constante de Boltzmann igual a la unidad $k_B = 1$.

Inicialmente se usarán condiciones de contorno periódicas en ambas direcciones para ver la evolución del sistema y partiendo con varios valores de la magnetización. Sin embargo, para facilitar la observación de los dominios magnéticos, pondremos condiciones de contorno periódicas sólo en la dirección x mientras que fijaremos los espines en los bordes de la dirección y .

2. Material utilizado.

Para realizar las simulaciones, aparte del cluster del departamento de física estadística (JOEL), se utilizó un ordenador con las siguientes especificaciones:

- Procesador: Intel (R) Core(TM) i7-10750H CPU @ 2.60GHz (2.59 GHz)
- Tarjeta gráfica: NVIDIA GeForce GTX 1650 with Max Q Design

En cuanto a software, el código se escribió íntegramente en VSCode, utilizando el lenguaje C++ para realizar la simulación y scripts de python para graficar y analizar los resultados. Asimismo, se utilizaron LLMs para agilizar la programación y la búsqueda de errores, en concreto el Copilot integrado en VSCode y Gemini.

Código.

Para realizar la simulación se utilizaron dos versiones de un programa en C++, una sin optimizar y otra paralelizada a través de OpenMP. Para realizar la paralelización, se utilizó el chat de Copilot integrado en VSCode. El resto del código se programó a partir

del código utilizado en el problema obligatorio del modelo de Ising, con modificaciones según el enunciado del problema.

Código sin optimizar:

```
#include <iostream>
#include <cmath>
#include <chrono>
#include <cstdlib>
#include <vector> // Necesario para almacenar las mediciones

#define N 128 // Tamaño de las matrices
#define TOTAL_TRIALS 100000000

using namespace std;

static int sord[N][N];
static int sdesord[N][N];
static int sord_next[N][N];
static int sdesord_next[N][N];

// === FORWARD DECLARATIONS ===
void export_density_profile(FILE* file, const int spins[N][N],
int L, double T);
int delta_energy_swap_L(const int spins[128][128], int n1, int
m1, int n2, int m2, int L);
int total_energy_L(const int spins[128][128], int L);
void select_neighbor_bc(int n, int m, int L, int& n2, int& m2);
void copy_lattice(const int src[128][128], int dst[128][128], int
L);

// -----
// NUEVA FUNCIÓN: Cálculo de errores estadísticos (Montecarlo)
// -----
void calcular_error_montecarlo(const std::vector<double>&
medidas, double& media, double& error)
{
    int N_exp = medidas.size();
    media = 0.0;

    // 1. Calculamos el valor medio
    for (int i = 0; i < N_exp; ++i)
    {
        media += medidas[i];
    }
    media /= N_exp;

    // 2. Calculamos la varianza ( $\sigma^2 = 1/N * \sum (X_i - X_{media})^2$ )
    double varianza = 0.0;
    for (int i = 0; i < N_exp; ++i)
    {
        varianza += (medidas[i] - media) * (medidas[i] - media);
    }
    varianza /= N_exp;
```

```
// 3. Calculamos el error (sigma / sqrt(N)) con un 68.26% de
probabilidad
    error = sqrt(varianza / N_exp);
}

// Devuelve un número aleatorio uniforme en [0, 1)
double random_double()
{
    return static_cast<double>(rand()) / (RAND_MAX + 1.0);
}

// Calcula el cambio de energía para el espín en la posición (n,
m)
int delta_energy(const int spins[N][N], int n, int m)
{
    int up = spins[(n + 1) % N][m];
    int down = spins[(n - 1 + N) % N][m];
    int right = spins[n][(m + 1) % N];
    int left = spins[n][(m - 1 + N) % N];
    return 2 * spins[n][m] * (up + down + right + left);
}

// Calcula el cambio de energía al intercambiar dos espines en
(n1,m1) y (n2,m2)
// Versión parametrizada por tamaño L
int delta_energy_swap_L(const int spins[128][128], int n1, int
m1, int n2, int m2, int L)
{
    if (n1 == n2 && m1 == m2) return 0;

    int up1 = spins[(n1 + 1) % L][m1];
    int down1 = spins[(n1 - 1 + L) % L][m1];
    int right1 = spins[n1][(m1 + 1) % L];
    int left1 = spins[n1][(m1 - 1 + L) % L];
    int sum1 = up1 + down1 + right1 + left1;

    int up2 = spins[(n2 + 1) % L][m2];
    int down2 = spins[(n2 - 1 + L) % L][m2];
    int right2 = spins[n2][(m2 + 1) % L];
    int left2 = spins[n2][(m2 - 1 + L) % L];
    int sum2 = up2 + down2 + right2 + left2;

    return (spins[n1][m1] - spins[n2][m2]) * (sum1 - sum2);
}

// Versión para matriz de tamaño fijo N (para mantener
compatibilidad con código existente)
int delta_energy_swap(const int spins[N][N], int n1, int m1, int
n2, int m2)
{
    return delta_energy_swap_L((const int (*)(128))spins, n1, m1,
n2, m2, N);
}

// Calcula la energía total del sistema en cada configuracion "a
```

```

lo bruto"
// Versión parametrizada por tamaño L
int total_energy_L(const int spins[128][128], int L)
{
    int energy = 0;
    for (int i = 0; i < L; ++i)
    {
        for (int j = 0; j < L; ++j)
        {
            energy += -spins[i][j] * (spins[(i + 1) % L][j] +
spins[(i - 1 + L) % L][j] + spins[i][(j + 1) % L] + spins[i][(j -
1 + L) % L]);
        }
    }
    return energy / 2; // Dividimos por 2 para evitar contar cada
interacción dos veces
}

// Versión para matriz de tamaño fijo N (para mantener
compatibilidad)
int total_energy(const int spins[N][N])
{
    return total_energy_L((const int (*)(128))spins, N);
}

// Inicializa la configuración ordenada (todos los espines +1)
void initialize_ordered(int spins[N][N])
{
    for (int n = 0; n < N; ++n)
    {
        for (int m = 0; m < N; ++m)
        {
            spins[n][m] = 1;
        }
    }
}

// Inicializa la configuración desordenada
void initialize_disordered(int spins[N][N])
{
    for (int n = 0; n < N; ++n)
    {
        for (int m = 0; m < N; ++m)
        {
            spins[n][m] = (random_double() <= 0.5) ? 1 : -1;
        }
    }
}

// Inicializa la configuración desordenada con condiciones de
frontera fijas en los bordes de la dirección X
void initialize_disordered_bc(int spins[N][N])
{
    for (int n = 0; n < N; ++n)
    {
        for (int m = 0; m < N; ++m)

```

```
{
    if (n == 0 || n == N - 1)
        spins[n][m] = 1; // Bordes fijos en el eje X
    else
        spins[n][m] = (random_double() <= 0.5) ? 1 : -1;
}
}

// Copia una submatriz de tamaño L x L de src a dst
void copy_lattice(const int src[128][128], int dst[128][128], int
L)
{
    for (int n = 0; n < L; ++n)
    {
        for (int m = 0; m < L; ++m)
        {
            dst[n][m] = src[n][m];
        }
    }
}

// Selecciona un vecino válido con condiciones de contorno fijas
en los bordes X
// Retorna las coordenadas en n2 y m2
void select_neighbor_bc(int n, int m, int L, int& n2, int& m2)
{
    int candidates[4][2];
    int count = 0;

    // Vecino derecha, solo si no es borde fijo
    if (n + 1 != L - 1)
    {
        candidates[count][0] = n + 1;
        candidates[count][1] = m;
        count++;
    }

    // Vecino izquierda, solo si no es borde fijo
    if (n - 1 != 0)
    {
        candidates[count][0] = n - 1;
        candidates[count][1] = m;
        count++;
    }

    // Vecino arriba
    candidates[count][0] = n;
    candidates[count][1] = (m + 1) % L;
    count++;

    // Vecino abajo
    candidates[count][0] = n;
    candidates[count][1] = (m - 1 + L) % L;
    count++;
}
```

```

        int choice = rand() % count;
        n2 = candidates[choice][0];
        m2 = candidates[choice][1];
    }

    //
    -----
    // NUEVA FUNCIÓN: Inicialización desordenada para una
    magnetización fija  $m_0$  (Kawasaki)
    //
    -----

    void initialize_kawasaki_m0(int spins[128][128], double m0, int
    L)
    {
        // 1. Inicializamos toda la submatriz útil de tamaño  $L \times L$ 
        con espines -1.
        // Esto limpia el sistema y establece el estado base antes de
        añadir los espines +1.
        for (int n = 0; n < L; ++n)
        {
            for (int m = 0; m < L; ++m)
            {
                if (n < 128 && m < 128) // Verificación de seguridad
                    spins[n][m] = -1;
            }
        }

        // 2. Relación teórica entre magnetización por partícula ( $m_0$ )
        y la fracción 'x' de espines +1:
        //  $m_0 = (N_{plus} - N_{minus}) / (L \cdot L)$ 
        // Sabiendo que  $N_{plus} + N_{minus} = L \cdot L$ , llegamos a que la
        fracción es  $x = (1 + m_0) / 2$ .
        double x = (1.0 + m0) / 2.0;
        int total_spins = L * L;

        // Calculamos el número exacto de espines +1 necesarios.
        // Usamos round() de <cmath> para aproximar al entero más
        cercano debido a la discretización de la red.
        int count_plus_needed = static_cast<int>(round(x *
        total_spins));

        // 3. Distribuimos aleatoriamente los espines +1 calculados
        en la red.
        // Se utiliza el mismo algoritmo de muestreo por rechazo que
        implementaste en calculate_magnetization().
        int count_ones = 0;
        while (count_ones < count_plus_needed)
        {
            int rn = rand() % L;
            int rm = rand() % L;

            // Si la posición elegida al azar contiene un -1, la
            cambiamos a +1
            if (spins[rn][rm] == -1)
            {
                spins[rn][rm] = 1;
            }
        }
    }

```

```
        count_ones++; // Incrementamos el contador solo si se
// ha modificado un espín de forma efectiva
    }
}

// Escribe la matriz de espines en el fichero con formato CSV
void write_lattice(FILE *file, const int spins[N][N])
{
    for (int n = 0; n < N; ++n)
    {
        for (int m = 0; m < N; ++m)
        {
            if (m < N - 1)
                fprintf(file, "%i,", spins[n][m]);
            else
                fprintf(file, "%i", spins[n][m]);
        }
        fprintf(file, "\n");
    }
    fprintf(file, "\n");
}

// Realiza un paso de Monte Carlo completo (un "sweep" N*N de
// ensayos) utilizando la dinámica de Kawasaki
void metropolis_sweep(int spins[N][N], int spins_next[N][N],
double T)
{
    for (int trial = 0; trial < N * N; ++trial)
    {
        int n = rand() % N;
        int m = rand() % N;

        // Elegimos espín vecino aleatorio para intercambiarlo con
// el espín actual
        int n2 = n;
        int m2 = m;
        int vecino = rand() % 4;
        if (vecino == 0)
            n2 = (n + 1) % N;
        else if (vecino == 1)
            n2 = (n - 1 + N) % N;
        else if (vecino == 2)
            m2 = (m + 1) % N;
        else
            m2 = (m - 1 + N) % N;
        // Calculamos el cambio de energía localmente (O(1)) y
// decidimos aceptación
        int dE = delta_energy_swap(spins, n, m, n2, m2);
        double p = min(1.0, exp(-dE / T));
        if (random_double() < p)
        {
            int temp = spins[n][m];
            spins[n][m] = spins[n2][m2];
            spins[n2][m2] = temp;
            spins_next[n][m] = spins[n][m];
        }
    }
}
```



```

        spins_next[n2][m2] = spins[n2][m2];
    }
    else
    {
        spins_next[n][m] = spins[n][m];
        spins_next[n2][m2] = spins[n2][m2];
    }
}

// Realiza un paso de Monte Carlo completo (un "sweep" N*N de
// ensayos) utilizando la dinámica de Kawasaki
// utilizando las condiciones de contorno fijas en los bordes de
// la dirección X
void metropolis_sweep_bc(int spins[N][N], int spins_next[N][N],
double T)
{
    for (int trial = 0; trial < N * N; ++trial)
    {
        // Elegimos un espín aleatorio que no esté en los bordes
        fijos
        int n = 1 + rand() % (N - 2); // solo posiciones
        interiores en X
        int m = rand() % N;

        // Elegimos un vecino aleatorio válido que no esté en los
        bordes fijos
        int candidates[4][2];
        int count = 0;

        // Vecino derecha, solo si no es borde fijo
        if (n + 1 != N - 1)
        {
            candidates[count][0] = n + 1;
            candidates[count][1] = m;
            count++;
        }

        // Vecino izquierda, solo si no es borde fijo
        if (n - 1 != 0)
        {
            candidates[count][0] = n - 1;
            candidates[count][1] = m;
            count++;
        }

        // Vecino arriba
        candidates[count][0] = n;
        candidates[count][1] = (m + 1) % N;
        count++;

        // Vecino abajo
        candidates[count][0] = n;
        candidates[count][1] = (m - 1 + N) % N;
        count++;
    }
}

```

```

        int choice = rand() % count;
        int n2 = candidates[choice][0];
        int m2 = candidates[choice][1];

        // Calculamos el cambio de energía localmente (O(1)) y
        decidimos aceptación
        int dE = delta_energy_swap(spins, n, m, n2, m2);
        double p = min(1.0, exp(-dE / T));
        if (random_double() < p)
        {
            int temp = spins[n][m];
            spins[n][m] = spins[n2][m2];
            spins[n2][m2] = temp;
            spins_next[n][m] = spins[n][m];
            spins_next[n2][m2] = spins[n2][m2];
        }
        else
        {
            spins_next[n][m] = spins[n][m];
            spins_next[n2][m2] = spins[n2][m2];
        }
    }
}

// Calcula y almacena las curvas de magnetización para diferentes
temperaturas
void calculate_magnetization()
{
    #define N_MAG 128
    int sN[N_MAG][N_MAG];
    constexpr int N_TEMPS = 10;
    const double T_m[N_TEMPS] = {1.5, 1.7, 1.9, 2.1, 2.3, 2.5,
2.7, 2.9, 3.1, 3.3};
    int i, j, k, n, m;

    // === CAMBIO 1: Eliminamos E, E2, M2 globales de aquí. ===
    // Por qué: Deben inicializarse por separado para cada
    repetición y tamaño,
    // así que usaremos vectores dentro del bucle como
    propusiste.

    FILE *mag32 = fopen("magn32.txt", "w");
    FILE *mag64 = fopen("magn64.txt", "w");
    FILE *mag128 = fopen("magn128.txt", "w");
    FILE *prof32 = fopen("perfil32.txt", "w");
    FILE *prof64 = fopen("perfil64.txt", "w");
    FILE *prof128 = fopen("perfil128.txt", "w");
    FILE *files_prof[3] = {prof32, prof64, prof128};

    if (!mag32 || !mag64 || !mag128)
    {
        std::cerr << "Error abriendo archivos de magnetización."
<< std::endl;
        return;
    }
}

```

```

    // Número de experimentos independientes por temperatura para
    sacar estadísticas
    const int N_experimentos = 10;

    int sizes[3] = {32, 64, 128};
    FILE *files[3] = {mag32, mag64, mag128};

    // Hacemos los cálculos de la magnetización
    for (i = 0; i < N_TEMPS; i++)
    {
        for (int s_idx = 0; s_idx < 3; ++s_idx)
        {
            int L = sizes[s_idx];

            // === CAMBIO 3: Adaptamos los vectores para E, E2 y
M2 ===
            // Por qué: Para almacenar el promedio de E, E2 y M2
            de CADA repetición independiente.
            std::vector<double> medidas_M(N_experimentos);
            std::vector<double> medidas_E(N_experimentos);
            std::vector<double> medidas_E2(N_experimentos);
            std::vector<double> medidas_M2(N_experimentos);

            // Repetimos el experimento N veces para calcular los
errores
            for (int rep = 0; rep < N_experimentos; ++rep)
            {
                for (n = 0; n < L; n++)
                {
                    for (m = 0; m < L; m++)
                    {
                        sN[n][m] = -1;
                    }
                }

                int half_spins = (L * L) / 2;
                int count_ones = 0;
                while (count_ones < half_spins)
                {
                    int rn = rand() % L;
                    int rm = rand() % L;
                    if (sN[rn][rm] == -1)
                    {
                        sN[rn][rm] = 1;
                        count_ones++;
                    }
                }

                int sN_next[N_MAG][N_MAG];
                for (n = 0; n < L; ++n)
                {
                    for (m = 0; m < L; ++m)
                    {
                        sN_next[n][m] = sN[n][m];
                    }
                }
            }
        }
    }

```

```

const int total_mc_trials = 1000000;
int mc_steps = total_mc_trials / (L * L);
int extra_trials = total_mc_trials % (L * L);

// === CAMBIO 4: Variables termodinámicas para
ESTA repetición concreta ===
double suma_E = 0.0;
double suma_E2 = 0.0;
int numero_medidas = 0; // Para promediar al
final de la repetición

// Calculamos la energía base SOLO UNA VEZ por
repetición (Rendimiento O(1) en el bucle)
double energia_actual = total_energy_L(sN, L);

for (int step = 0; step < mc_steps; ++step)
{
    for (int trial = 0; trial < L * L; ++trial)
    {
        int n = 1 + rand() % (L - 2);
        int m = rand() % L;

        int n2, m2;
        select_neighbor_bc(n, m, L, n2, m2);

        int dE = delta_energy_swap_L(sN, n, m,
n2, m2, L);

        double p = min(1.0, exp(-dE / T_m[i]));
        if (random_double() < p)
        {
            int temp = sN[n][m];
            sN[n][m] = sN[n2][m2];
            sN[n2][m2] = temp;
            sN_next[n][m] = sN[n][m];
            sN_next[n2][m2] = sN[n2][m2];

            // === CAMBIO 4 (Continuación):
Actualizamos la energía en O(1) ===
            energia_actual += dE;
        }
    }
    // Acumulamos la medida tras cada sweep de
Montecarlo

    suma_E += energia_actual;
    suma_E2 += (energia_actual * energia_actual);
    numero_medidas++;
}

if (extra_trials > 0)
{
    for (j = 0; j < extra_trials; ++j)
    {
        int n = 1 + rand() % (L - 2);
        int m = rand() % L;

```

```

        int n2, m2;
        select_neighbor_bc(n, m, L, n2, m2);

        int dE = delta_energy_swap_L(sN, n, m,
n2, m2, L);

        double p = min(1.0, exp(-dE / T_m[i]));
        if (random_double() < p)
        {
            int temp = sN[n][m];
            sN[n][m] = sN[n2][m2];
            sN[n2][m2] = temp;
            sN_next[n][m] = sN[n][m];
            sN_next[n2][m2] = sN[n2][m2];

            // === CAMBIO 4 (Continuación) ===
            energia_actual += dE;
        }
    }
    suma_E += energia_actual;
    suma_E2 += (energia_actual * energia_actual);
    numero_medidas++;
}

double m_top = 0.0;
double m_bottom = 0.0;

for (j = 0; j < L; ++j)
{
    for (k = 0; k < L; ++k)
    {
        if (k < L / 2)
        {
            m_top += sN[j][k];
        }
        else
        {
            m_bottom += sN[j][k];
        }
    }
}

double particles_per_half = (L * L) / 2.0;
double mag_domain = (fabs(m_top) /
particles_per_half + fabs(m_bottom) / particles_per_half) / 2.0;

// === CAMBIO 1.2: Normalización por partícula
===

// Convertimos la energía total acumulada a
energía por partícula dividiendo entre N (que es L*L).
// Nota: La energía al cuadrado (E2) se debe
dividir por el cuadrado del número de partículas ((L*L)^2).
double num_particulas = L * L;

medidas_M[rep] = mag_domain;
medidas_M2[rep] = (mag_domain * mag_domain);
medidas_E[rep] = (suma_E / numero_medidas) /

```

```

num_particulas;
    medidas_E2[rep] = (suma_E2 / numero_medidas) /
(num_particulas * num_particulas);

    // Solo necesitamos el perfil de una de las
    // repeticiones (por ejemplo, la última)
    // para tener una "foto" representativa del
    // estado del sistema a esta temperatura.
    if (rep == N_experimentos - 1)
    {
        // casteamos sN a const int (*)[N_MAG] para
        // que coincida con la firma de la función
        export_density_profile(files_prof[s_idx],
        (const int (*)[N_MAG])sN, L, T_m[i]);
    }
}

// === CAMBIO 6: Calculamos medias y errores para
// todas las magnitudes termodinámicas ===
double media_M, err_M;
calcular_error_montecarlo(medidas_M, media_M, err_M);

double media_M2, err_M2;
calcular_error_montecarlo(medidas_M2, media_M2,
err_M2);

double media_E, err_E;
calcular_error_montecarlo(medidas_E, media_E, err_E);

double media_E2, err_E2;
calcular_error_montecarlo(medidas_E2, media_E2,
err_E2);

// === CAMBIO 1.3: Cálculo de calor específico y
// susceptibilidad ===
// Como las medias que hemos obtenido están "por
// partícula" ( $\langle e \rangle$ ,  $\langle e^2 \rangle$ ,  $\langle m \rangle$ ,  $\langle m^2 \rangle$ ),
// aplicamos el factor multiplicativo ( $L^2$ ) derivado
// de adaptar la fórmula extensiva del PDF a valores intensivos.
double num_particulas = L * L;
double T = T_m[i];

double c_N = (num_particulas / (T * T)) * (media_E2 -
(media_E * media_E));
double chi_N = (num_particulas / T) * (media_M2 -
(media_M * media_M));

// === CAMBIO 1.3 (Parte 2): Añadidas c_N y chi_N al
// archivo de salida ===
// Formato CSV por tabulaciones: T |  $\langle M \rangle$  | err_M |
//  $\langle E \rangle$  | err_E | c_N | chi_N
fprintf(files[s_idx],
"%lf\t%e\t%e\t%e\t%e\t%e\n", T_m[i], media_M, err_M, media_E,
err_E, c_N, chi_N);
}
}

```

```
    fclose(mag32);
    fclose(mag64);
    fclose(mag128);
}

// -----
// NUEVA FUNCIÓN: Conteo global de espines (+1 y -1)
// -----
void export_global_spin_count(FILE* file, const int spins[128]
[128], int L, double T)
{
    int count_plus = 0;
    int count_minus = 0;

    for (int i = 0; i < L; ++i)
    {
        for (int j = 0; j < L; ++j)
        {
            if (spins[i][j] == 1)
            {
                count_plus++;
            }
            else if (spins[i][j] == -1)
            {
                count_minus++;
            }
        }
    }

    // Formato CSV por tabulaciones: Temperatura | Cantidad +1 |
    Cantidad -1
    fprintf(file, "%lf\t%d\t%d\n", T, count_plus, count_minus);
}

// -----
// NUEVA FUNCIÓN: Perfil de densidad fila por fila
// -----
void export_density_profile(FILE* file, const int spins[N][N],
int L, double T)
{
    // Iteramos sobre la coordenada Y (que en tu matriz es el
    segundo índice, 'm')
    for (int m = 0; m < L; ++m)
    {
        int count_plus = 0;
        int count_minus = 0;

        // Sumamos todos los valores a lo largo de la coordenada
        X (primer índice, 'n')
        for (int n = 0; n < L; ++n)
        {
            if (spins[n][m] == 1)
                count_plus++;
            else if (spins[n][m] == -1)
                count_minus++;
        }
    }
}
```

```

    }

    // Exportamos: Temperatura, Coordenada Y, Cantidad +1,
    Cantidad -1
    fprintf(file, "%lf\t%d\t%d\t%d\n", T, m, count_plus,
count_minus);
    }
    // Dejamos una línea en blanco al terminar la matriz.
    // Esto es muy útil en Gnuplot para separar los datos de
    distintas temperaturas.
    fprintf(file, "\n");
}

//
-----
// NUEVA FUNCIÓN: Calcula magnetización para  $m_0$  distinto de cero
//
-----
void calculate_magnetization_m0(double m0)
{
    #define N_MAG 128
    int sN[N_MAG][N_MAG];
    constexpr int N_TEMPS = 10;
    const double T_m[N_TEMPS] = {1.5, 1.7, 1.9, 2.1, 2.3, 2.5,
2.7, 2.9, 3.1, 3.3};
    int i, j, k, n, m;

    // Abrimos archivos con sufijo _m0 para no sobrescribir los
    datos de  $m_0=0$ 
    FILE *mag32 = fopen("magn32_m0.txt", "w");
    FILE *mag64 = fopen("magn64_m0.txt", "w");
    FILE *mag128 = fopen("magn128_m0.txt", "w");
    FILE *prof32 = fopen("perfil32_m0.txt", "w");
    FILE *prof64 = fopen("perfil64_m0.txt", "w");
    FILE *prof128 = fopen("perfil128_m0.txt", "w");

    // === CAMBIO: Se ha eliminado la declaración global de
    f_evol aquí.
    // Ahora se creará de forma dinámica dentro de los bucles.

    FILE *files[3] = {mag32, mag64, mag128};
    FILE *files_prof[3] = {prof32, prof64, prof128};

    if (!mag32 || !mag64 || !mag128)
    {
        std::cerr << "Error abriendo archivos de magnetización
m0." << std::endl;
        return;
    }

    const int N_experimentos = 10;

    int sizes[3] = {32, 64, 128};

    // Calculamos la fracción teórica x
    double x_frac = (1.0 + m0) / 2.0;

```



```

for (i = 0; i < N_TEMPS; i++)
{
    for (int s_idx = 0; s_idx < 3; ++s_idx)
    {
        int L = sizes[s_idx];

        int L_split = static_cast<int>(round(x_frac * L));

        std::vector<double> medidas_M(N_experimentos);
        std::vector<double> medidas_E(N_experimentos);
        std::vector<double> medidas_E2(N_experimentos);
        std::vector<double> medidas_M2(N_experimentos);

        for (int rep = 0; rep < N_experimentos; ++rep)
        {
            initialize_kawasaki_m0((int (*)(N_MAG))sN, m0,
L);

            int sN_next[N_MAG][N_MAG];
            copy_lattice(sN, sN_next, L);

            const int total_mc_trials = 1000000;
            int mc_steps = total_mc_trials / (L * L);
            int extra_trials = total_mc_trials % (L * L);

            // === CAMBIO: Modificamos la condición para que
capture EN TODAS las temperaturas ===
            // Quitamos el (i == 0). Ahora captura la primera
repetición de la red más grande para cada T
            bool capture_evolution = (s_idx == 2 && rep ==
0);

            FILE *f_evol = nullptr;

            if (capture_evolution)
            {
                // === CAMBIO: Nombre dinámico que incluye
tanto m0 como la temperatura T ===
                char filename_evol[100];
                snprintf(filename_evol,
sizeof(filename_evol), "ising_evol_m0_%.2f_T_%.2f.dat", m0,
T_m[i]);
                f_evol = fopen(filename_evol, "w");
            }

            int writes_evol = 0;
            int max_writes_evol = std::max(1, mc_steps / 10);
            int energy_threshold_evol = L * 2;
            double last_energy_evol = 0;

            double suma_E = 0.0;
            double suma_E2 = 0.0;
            int numero_medidas = 0;

            double energia_actual = total_energy_L(sN, L);

```

```

        // Escribimos el estado inicial justo antes del
        bucle de Montecarlo
        if (capture_evolution && f_evol)
        {
            write_lattice(f_evol, (const int (*)[N])sN);
            last_energy_evol = energia_actual;
        }

        for (int step = 0; step < mc_steps; ++step)
        {
            for (int trial = 0; trial < L * L; ++trial)
            {
                int n = 1 + rand() % (L - 2);
                int m = rand() % L;

                int n2, m2;
                select_neighbor_bc(n, m, L, n2, m2);

                int dE = delta_energy_swap_L(sN, n, m,
n2, m2, L);

                double p = min(1.0, exp(-dE / T_m[i]));
                if (random_double() < p)
                {
                    int temp = sN[n][m];
                    sN[n][m] = sN[n2][m2];
                    sN[n2][m2] = temp;
                    sN_next[n][m] = sN[n][m];
                    sN_next[n2][m2] = sN[n2][m2];

                    energia_actual += dE;
                }
            }

            // Evaluamos si toca exportar el fotograma
            if (capture_evolution && f_evol &&
writes_evol < max_writes_evol)
            {
                if (std::abs(energia_actual -
last_energy_evol) >= energy_threshold_evol)
                {
                    write_lattice(f_evol, (const int (*)
[N])sN);

                    last_energy_evol = energia_actual;
                    writes_evol++;
                }
            }

            suma_E += energia_actual;
            suma_E2 += (energia_actual * energia_actual);
            numero_medidas++;
        }

        if (extra_trials > 0)
        {
            for (j = 0; j < extra_trials; ++j)
            {

```

```

        int n = 1 + rand() % (L - 2);
        int m = rand() % L;

        int n2, m2;
        select_neighbor_bc(n, m, L, n2, m2);

        int dE = delta_energy_swap_L(sN, n, m,
n2, m2, L);

        double p = min(1.0, exp(-dE / T_m[i]));
        if (random_double() < p)
        {
            int temp = sN[n][m];
            sN[n][m] = sN[n2][m2];
            sN[n2][m2] = temp;
            sN_next[n][m] = sN[n][m];
            sN_next[n2][m2] = sN[n2][m2];

            energia_actual += dE;
        }
    }

    // Última comprobación tras los extra_trials
    if (capture_evolution && f_evol &&
writes_evol < max_writes_evol)
    {
        if (std::abs(energia_actual -
last_energy_evol) >= energy_threshold_evol)
        {
            write_lattice(f_evol, (const int (*)
[N])sN);

            last_energy_evol = energia_actual;
            writes_evol++;
        }
    }

    suma_E += energia_actual;
    suma_E2 += (energia_actual * energia_actual);
    numero_medidas++;
}

// === CAMBIO: Cerramos el archivo de la
evolución de esta temperatura ===
if (capture_evolution && f_evol)
{
    fclose(f_evol);
}

double m_domain1 = 0.0;
double m_domain2 = 0.0;

for (j = 0; j < L; ++j)
{
    for (k = 0; k < L; ++k)
    {
        if (k < L_split)
        {

```

```

        m_domain1 += sN[j][k];
    }
    else
    {
        m_domain2 += sN[j][k];
    }
}

double particles_1 = L_split * L;
double particles_2 = (L - L_split) * L;

double mag_1 = (particles_1 > 0) ?
(fabs(m_domain1) / particles_1) : 0.0;
double mag_2 = (particles_2 > 0) ?
(fabs(m_domain2) / particles_2) : 0.0;

double mag_domain;
if (particles_1 == 0) mag_domain = mag_2;
else if (particles_2 == 0) mag_domain = mag_1;
else mag_domain = (mag_1 + mag_2) / 2.0;

double num_particulas = L * L;

medidas_M[rep] = mag_domain;
medidas_M2[rep] = (mag_domain * mag_domain);
medidas_E[rep] = (suma_E / numero_medidas) /
num_particulas;
medidas_E2[rep] = (suma_E2 / numero_medidas) /
(num_particulas * num_particulas);

if (rep == N_experimentos - 1)
{
    export_density_profile(files_prof[s_idx],
(const int (*)(N_MAG))sN, L, T_m[i]);
}

double media_M, err_M;
calcular_error_montecarlo(medidas_M, media_M, err_M);

double media_M2, err_M2;
calcular_error_montecarlo(medidas_M2, media_M2,
err_M2);

double media_E, err_E;
calcular_error_montecarlo(medidas_E, media_E, err_E);

double media_E2, err_E2;
calcular_error_montecarlo(medidas_E2, media_E2,
err_E2);

double num_particulas = L * L;
double T = T_m[i];

double c_N = (num_particulas / (T * T)) * (media_E2 -

```

```

(media_E * media_E));
        double chi_N = (num_particulas / T) * (media_M2 -
(media_M * media_M));

        fprintf(files[s_idx],
"%lf\t%e\t%e\t%e\t%e\t%e\t%e\n", T_m[i], media_M, err_M, media_E,
err_E, c_N, chi_N);
    }
}

fclose(mag32); fclose(mag64); fclose(mag128);
fclose(prof32); fclose(prof64); fclose(prof128);
}

int main()
{
    auto inicio = chrono::high_resolution_clock::now();
    int semilla = 310809;
    srand(semilla);

    std::cout << "Iniciando simulacion del modelo de Ising
(Dinamica de Kawasaki)..." << std::endl;

    //
    =====
    // TAREA 1: Simular la dinámica y obtener fotogramas para
    diferentes temperaturas
    //
    =====

    std::cout << "\n[1/3] Realizando Tarea 1: Generando
fotogramas de la red..." << std::endl;

    double T_low = 1.0;
    double T_high = 3.5;

    int mc_steps = TOTAL_TRIALS / (N * N);
    int extra_trials = TOTAL_TRIALS % (N * N);
    int max_writes = std::max(1, mc_steps / 10);
    const int energy_threshold = N * 2;

    //
    -----
    // CASO A: T_baja y T_alta con CONDICIONES PERIÓDICAS EN
    AMBOS EJES
    //
    -----

    std::cout << "          -> Simulando con Condiciones
Periodicas..." << std::endl;

    FILE *f_per_ord_low = fopen("ising_per_ord_low.dat", "w");
    FILE *f_per_des_low = fopen("ising_per_des_low.dat", "w");
    FILE *f_per_ord_high = fopen("ising_per_ord_high.dat", "w");
    FILE *f_per_des_high = fopen("ising_per_des_high.dat", "w");

    if (!f_per_ord_low || !f_per_des_low || !f_per_ord_high || !
f_per_des_high) {

```

```

        std::cerr << "Error abriendo archivos para condiciones
periodicas." << std::endl; return 1;
    }

    // Inicializamos: Ordenada para T_low/T_high, Desordenada
    para T_low/T_high
    initialize_ordered(sord);
    initialize_disordered(sdesord);

    // Copiamos a las matrices "next"
    for (int n = 0; n < N; ++n) {
        for (int m = 0; m < N; ++m) {
            sord_next[n][m] = sord[n][m];
            sdesord_next[n][m] = sdesord[n][m];
        }
    }

    int last_E_ord = total_energy(sord);
    int last_E_des = total_energy(sdesord);
    int writes_ord = 0, writes_des = 0;

    // Bucle Montecarlo (Usando metropolis_sweep normal) para
    T_baja
    for (int step = 0; step < mc_steps; ++step) {
        metropolis_sweep(sord, sord_next, T_low);
        metropolis_sweep(sdesord, sdesord_next, T_low);

        if (writes_ord < max_writes &&
std::abs(total_energy(sord) - last_E_ord) >= energy_threshold) {
            write_lattice(f_per_ord_low, sord);
            last_E_ord = total_energy(sord); writes_ord++;
        }
        if (writes_des < max_writes &&
std::abs(total_energy(sdesord) - last_E_des) >= energy_threshold)
        {
            write_lattice(f_per_des_low, sdesord);
            last_E_des = total_energy(sdesord); writes_des++;
        }
    }

    // Reiniciamos matrices para probar T_alta con Periódicas
    initialize_ordered(sord);
    initialize_disordered(sdesord);
    for (int n = 0; n < N; ++n) {
        for (int m = 0; m < N; ++m) {
            sord_next[n][m] = sord[n][m];
            sdesord_next[n][m] = sdesord[n][m];
        }
    }

    last_E_ord = total_energy(sord); last_E_des =
total_energy(sdesord);
    writes_ord = 0; writes_des = 0;

    // Bucle Montecarlo (Usando metropolis_sweep normal) para
    T_alta

```

```

    for (int step = 0; step < mc_steps; ++step) {
        metropolis_sweep(sord, sord_next, T_high);
        metropolis_sweep(sdesord, sdesord_next, T_high);

        if (writes_ord < max_writes &&
std::abs(total_energy(sord) - last_E_ord) >= energy_threshold) {
            write_lattice(f_per_ord_high, sord);
            last_E_ord = total_energy(sord); writes_ord++;
        }
        if (writes_des < max_writes &&
std::abs(total_energy(sdesord) - last_E_des) >= energy_threshold)
        {
            write_lattice(f_per_des_high, sdesord);
            last_E_des = total_energy(sdesord); writes_des++;
        }
    }

    fclose(f_per_ord_low); fclose(f_per_des_low);
    fclose(f_per_ord_high); fclose(f_per_des_high);

    //
    -----
    // CASO B: T_baja y T_alta con BORDES FIJOS EN X (Para
    observar mejor los dominios)
    //
    -----

    std::cout << "          -> Simulando con Bordes Fijos (eje X)..."
<< std::endl;

    FILE *f_bc_ord_low = fopen("ising_bc_ord_low.dat", "w");
    FILE *f_bc_des_low = fopen("ising_bc_des_low.dat", "w");
    FILE *f_bc_ord_high = fopen("ising_bc_ord_high.dat", "w");
    FILE *f_bc_des_high = fopen("ising_bc_des_high.dat", "w");

    if (!f_bc_ord_low || !f_bc_des_low || !f_bc_ord_high || !
f_bc_des_high) {
        std::cerr << "Error abriendo archivos para bordes fijos."
<< std::endl; return 1;
    }

    // Inicializamos usando la función con bordes fijos
    initialize_ordered(sord);
    initialize_disordered_bc(sdesord);
    for (int n = 0; n < N; ++n) {
        for (int m = 0; m < N; ++m) {
            sord_next[n][m] = sord[n][m];
            sdesord_next[n][m] = sdesord[n][m];
        }
    }

    last_E_ord = total_energy(sord); last_E_des =
total_energy(sdesord);
    writes_ord = 0; writes_des = 0;

    // Bucle Montecarlo (Usando metropolis_sweep_bc) para T_baja

```

```

    for (int step = 0; step < mc_steps; ++step) {
        metropolis_sweep_bc(sord, sord_next, T_low);
        metropolis_sweep_bc(sdesord, sdesord_next, T_low);

        if (writes_ord < max_writes &&
std::abs(total_energy(sord) - last_E_ord) >= energy_threshold) {
            write_lattice(f_bc_ord_low, sord);
            last_E_ord = total_energy(sord); writes_ord++;
        }
        if (writes_des < max_writes &&
std::abs(total_energy(sdesord) - last_E_des) >= energy_threshold)
        {
            write_lattice(f_bc_des_low, sdesord);
            last_E_des = total_energy(sdesord); writes_des++;
        }
    }

    // Reiniciamos matrices para probar T_alta con bordes fijos
    initialize_ordered(sord);
    initialize_disordered_bc(sdesord);
    for (int n = 0; n < N; ++n) {
        for (int m = 0; m < N; ++m) {
            sord_next[n][m] = sord[n][m];
            sdesord_next[n][m] = sdesord[n][m];
        }
    }

    last_E_ord = total_energy(sord); last_E_des =
total_energy(sdesord);
    writes_ord = 0; writes_des = 0;

    // Bucle Montecarlo (Usando metropolis_sweep_bc) para T_alta
    for (int step = 0; step < mc_steps; ++step) {
        metropolis_sweep_bc(sord, sord_next, T_high);
        metropolis_sweep_bc(sdesord, sdesord_next, T_high);

        if (writes_ord < max_writes &&
std::abs(total_energy(sord) - last_E_ord) >= energy_threshold) {
            write_lattice(f_bc_ord_high, sord);
            last_E_ord = total_energy(sord); writes_ord++;
        }
        if (writes_des < max_writes &&
std::abs(total_energy(sdesord) - last_E_des) >= energy_threshold)
        {
            write_lattice(f_bc_des_high, sdesord);
            last_E_des = total_energy(sdesord); writes_des++;
        }
    }

    fclose(f_bc_ord_low); fclose(f_bc_des_low);
    fclose(f_bc_ord_high); fclose(f_bc_des_high);

    //
=====
    // TAREAS 2 a 7: Cálculos termodinámicos con magnetización

```



```

nula (m0 = 0)
//
=====
// Esta función encapsula todo: curvas de magnetización,
energía, calor específico,
// susceptibilidad y los perfiles de densidad en el eje Y.
std::cout << "\n[2/3] Realizando Tareas 2-7: Termodinamica
(m0 = 0)..." << std::endl;
calculate_magnetization();

//
=====
// TAREA 8: Cálculos termodinámicos partiendo de una
magnetización NO nula (m0 != 0)
//
=====
// Ejecutamos exactamente la misma termodinámica
(magnetización, E, c_N, chi_N, perfiles)
// pero inicializando el sistema con la fracción asimétrica
correspondiente a m0.
std::cout << "\n[3/3] Realizando Tarea 8: Termodinamica (m0 !
= 0)..." << std::endl;
double m0_deseada_1 = 0.5; // Probamos con un m0 del 50%
double m0_deseada_2 = 0.8; // Probamos con un m0 del 80%
calculate_magnetization_m0(m0_deseada_1);
calculate_magnetization_m0(m0_deseada_2);
std::cout << "\nSimulacion finalizada con exito. Revisa los
archivos de salida generados." << std::endl;

auto fin = chrono::high_resolution_clock::now();
chrono::duration<double, milli> tiempo_ejecucion = fin -
inicio;
cout << "El código multi-run completo tardó: " <<
tiempo_ejecucion.count() << " milisegundos." << endl;

return 0;
}

```

Código optimizado:

```

#include <iostream>
#include <cmath>
#include <cstdlib>
#include <vector> // Necesario para almacenar las mediciones
#include <omp.h>
#include <chrono>
#include <cstdio>
#include <ctime>

#define N 128 // Tamaño de las matrices
#define TOTAL_TRIALS 100000000

using namespace std;

static int sord[N][N];
static int sdesord[N][N];

```

```

static int sord_next[N][N];
static int sdesord_next[N][N];

// === FORWARD DECLARATIONS ===
void export_density_profile(FILE* file, const int spins[N][N],
int L, double T);
int delta_energy_swap_L(const int spins[128][128], int n1, int
m1, int n2, int m2, int L);
int total_energy_L(const int spins[128][128], int L);
void select_neighbor_bc(int n, int m, int L, int& n2, int& m2);
void copy_lattice(const int src[128][128], int dst[128][128], int
L);

// -----
// NUEVA FUNCIÓN: Cálculo de errores estadísticos (Montecarlo)
// -----
void calcular_error_montecarlo(const std::vector<double>&
medidas, double& media, double& error)
{
    int N_exp = medidas.size();
    media = 0.0;

    // 1. Calculamos el valor medio
    for (int i = 0; i < N_exp; ++i)
    {
        media += medidas[i];
    }
    media /= N_exp;

    // 2. Calculamos la varianza ( $\sigma^2 = 1/N * \sum(X_i - X_{media})^2$ )
    double varianza = 0.0;
    for (int i = 0; i < N_exp; ++i)
    {
        varianza += (medidas[i] - media) * (medidas[i] - media);
    }
    varianza /= N_exp;

    // 3. Calculamos el error ( $\sigma / \sqrt{N}$ ) con un 68.26% de
    probabilidad
    error = sqrt(varianza / N_exp);
}

// Devuelve un número aleatorio uniforme en [0, 1)
double random_double()
{
    return static_cast<double>(rand()) / (RAND_MAX + 1.0);
}

// Thread-safe RNG using rand_r
inline double random_double_r(unsigned int &seed)
{
    return static_cast<double>(rand_r(&seed)) / (RAND_MAX + 1.0);
}

// Calcula el cambio de energía para el espín en la posición (n,

```

```

m)
int delta_energy(const int spins[N][N], int n, int m)
{
    int up = spins[(n + 1) % N][m];
    int down = spins[(n - 1 + N) % N][m];
    int right = spins[n][(m + 1) % N];
    int left = spins[n][(m - 1 + N) % N];
    return 2 * spins[n][m] * (up + down + right + left);
}

// Calcula el cambio de energía al intercambiar dos espines en
// (n1,m1) y (n2,m2)
// Versión parametrizada por tamaño L
int delta_energy_swap_L(const int spins[128][128], int n1, int
n1, int n2, int m2, int L)
{
    if (n1 == n2 && m1 == m2) return 0;

    int up1 = spins[(n1 + 1) % L][m1];
    int down1 = spins[(n1 - 1 + L) % L][m1];
    int right1 = spins[n1][(m1 + 1) % L];
    int left1 = spins[n1][(m1 - 1 + L) % L];
    int sum1 = up1 + down1 + right1 + left1;

    int up2 = spins[(n2 + 1) % L][m2];
    int down2 = spins[(n2 - 1 + L) % L][m2];
    int right2 = spins[n2][(m2 + 1) % L];
    int left2 = spins[n2][(m2 - 1 + L) % L];
    int sum2 = up2 + down2 + right2 + left2;

    return (spins[n1][m1] - spins[n2][m2]) * (sum1 - sum2);
}

// Versión para matriz de tamaño fijo N (para mantener
// compatibilidad con código existente)
int delta_energy_swap(const int spins[N][N], int n1, int m1, int
n2, int m2)
{
    return delta_energy_swap_L((const int (*)[128])spins, n1, m1,
n2, m2, N);
}

// Calcula la energía total del sistema en cada configuración "a
// lo bruto"
// Versión parametrizada por tamaño L
int total_energy_L(const int spins[128][128], int L)
{
    int energy = 0;
    #pragma omp parallel for reduction(+:energy) schedule(static)
    for (int i = 0; i < L; ++i)
    {
        for (int j = 0; j < L; ++j)
        {
            energy += -spins[i][j] * (spins[(i + 1) % L][j] +
spins[(i - 1 + L) % L][j] + spins[i][(j + 1) % L] + spins[i][(j -
1 + L) % L]);
        }
    }
}

```

```
    }
}

    return energy / 2; // Dividimos por 2 para evitar contar cada
interacción dos veces
}

// Versión para matriz de tamaño fijo N (para mantener
compatibilidad)
int total_energy(const int spins[N][N])
{
    return total_energy_L((const int (*)[128])spins, N);
}

// Inicializa la configuración ordenada (todos los espines +1)
void initialize_ordered(int spins[N][N])
{
    #pragma omp parallel for collapse(2) schedule(static)
    for (int n = 0; n < N; ++n)
    {
        for (int m = 0; m < N; ++m)
        {
            spins[n][m] = 1;
        }
    }
}

// Inicializa la configuración desordenada
void initialize_disordered(int spins[N][N])
{
    #pragma omp parallel
    {
        unsigned int seed = (unsigned int)time(NULL) ^ (unsigned
int)omp_get_thread_num();
        #pragma omp for collapse(2) schedule(static)
        for (int n = 0; n < N; ++n)
        {
            for (int m = 0; m < N; ++m)
            {
                spins[n][m] = (random_double_r(seed) <= 0.5) ? 1
: -1;
            }
        }
    }
}

// Inicializa la configuración desordenada con condiciones de
frontera fijas en los bordes de la dirección X
void initialize_disordered_bc(int spins[N][N])
{
    #pragma omp parallel
    {
        unsigned int seed = (unsigned int)time(NULL) ^ (unsigned
int)omp_get_thread_num();
        #pragma omp for collapse(2) schedule(static)
        for (int n = 0; n < N; ++n)
        {
```

```
        for (int m = 0; m < N; ++m)
        {
            if (n == 0 || n == N - 1)
                spins[n][m] = 1; // Bordes fijos en el eje X
            else
                spins[n][m] = (random_double_r(seed) <= 0.5)
? 1 : -1;
        }
    }
}

// Copia una submatriz de tamaño L x L de src a dst
void copy_lattice(const int src[128][128], int dst[128][128], int
L)
{
    #pragma omp parallel for schedule(static)
    for (int n = 0; n < L; ++n)
    {
        for (int m = 0; m < L; ++m)
        {
            dst[n][m] = src[n][m];
        }
    }
}

// Selecciona un vecino válido con condiciones de contorno fijas
en los bordes X
// Retorna las coordenadas en n2 y m2
void select_neighbor_bc(int n, int m, int L, int& n2, int& m2)
{
    int candidates[4][2];
    int count = 0;

    // Vecino derecha, solo si no es borde fijo
    if (n + 1 != L - 1)
    {
        candidates[count][0] = n + 1;
        candidates[count][1] = m;
        count++;
    }

    // Vecino izquierda, solo si no es borde fijo
    if (n - 1 != 0)
    {
        candidates[count][0] = n - 1;
        candidates[count][1] = m;
        count++;
    }

    // Vecino arriba
    candidates[count][0] = n;
    candidates[count][1] = (m + 1) % L;
    count++;

    // Vecino abajo
```

```

    candidates[count][0] = n;
    candidates[count][1] = (m - 1 + L) % L;
    count++;

    int choice = rand() % count;
    n2 = candidates[choice][0];
    m2 = candidates[choice][1];
}

//
-----
// NUEVA FUNCIÓN: Inicialización desordenada para una
// magnetización fija m0 (Kawasaki)
//
-----
void initialize_kawasaki_m0(int spins[128][128], double m0, int
L)
{
    // 1. Inicializamos toda la submatriz útil de tamaño L x L
    // con espines -1.
    // Esto limpia el sistema y establece el estado base antes de
    // añadir los espines +1.
    for (int n = 0; n < L; ++n)
    {
        for (int m = 0; m < L; ++m)
        {
            if (n < 128 && m < 128) // Verificación de seguridad
                spins[n][m] = -1;
        }
    }

    // 2. Relación teórica entre magnetización por partícula (m0)
    // y la fracción 'x' de espines +1:
    //  $m0 = (N_{plus} - N_{minus}) / (L*L)$ 
    // Sabiendo que  $N_{plus} + N_{minus} = L*L$ , llegamos a que la
    // fracción es  $x = (1 + m0) / 2$ .
    double x = (1.0 + m0) / 2.0;
    int total_spins = L * L;

    // Calculamos el número exacto de espines +1 necesarios.
    // Usamos round() de <cmath> para aproximar al entero más
    // cercano debido a la discretización de la red.
    int count_plus_needed = static_cast<int>(round(x *
total_spins));

    // 3. Distribuimos aleatoriamente los espines +1 calculados
    // en la red.
    // Se utiliza el mismo algoritmo de muestreo por rechazo que
    // implementaste en calculate_magnetization().
    int count_ones = 0;
    while (count_ones < count_plus_needed)
    {
        int rn = rand() % L;
        int rm = rand() % L;

        // Si la posición elegida al azar contiene un -1, la

```

```

cambiamos a +1
    if (spins[rn][rm] == -1)
    {
        spins[rn][rm] = 1;
        count_ones++; // Incrementamos el contador solo si se
// ha modificado un espín de forma efectiva
    }
}

// Escribe la matriz de espines en el fichero con formato CSV
void write_lattice(FILE *file, const int spins[N][N])
{
    for (int n = 0; n < N; ++n)
    {
        for (int m = 0; m < N; ++m)
        {
            if (m < N - 1)
                fprintf(file, "%i,", spins[n][m]);
            else
                fprintf(file, "%i", spins[n][m]);
        }
        fprintf(file, "\n");
    }
    fprintf(file, "\n");
}

// Realiza un paso de Monte Carlo completo (un "sweep" N*N de
// ensayos) utilizando la dinámica de Kawasaki
void metropolis_sweep(int spins[N][N], int spins_next[N][N],
double T)
{
    for (int trial = 0; trial < N * N; ++trial)
    {
        int n = rand() % N;
        int m = rand() % N;

        // Elegimos espín vecino aleatorio para intercambiarlo con
// el espín actual
        int n2 = n;
        int m2 = m;
        int vecino = rand() % 4;
        if (vecino == 0)
            n2 = (n + 1) % N;
        else if (vecino == 1)
            n2 = (n - 1 + N) % N;
        else if (vecino == 2)
            m2 = (m + 1) % N;
        else
            m2 = (m - 1 + N) % N;
        // Calculamos el cambio de energía localmente (O(1)) y
// decidimos aceptación
        int dE = delta_energy_swap(spins, n, m, n2, m2);
        double p = min(1.0, exp(-dE / T));
        if (random_double() < p)
        {

```

```
        int temp = spins[n][m];
        spins[n][m] = spins[n2][m2];
        spins[n2][m2] = temp;
        spins_next[n][m] = spins[n][m];
        spins_next[n2][m2] = spins[n2][m2];
    }
    else
    {
        spins_next[n][m] = spins[n][m];
        spins_next[n2][m2] = spins[n2][m2];
    }
}

// Realiza un paso de Monte Carlo completo (un "sweep" N*N de
// ensayos) utilizando la dinámica de Kawasaki
// utilizando las condiciones de contorno fijas en los bordes de
// la dirección X
void metropolis_sweep_bc(int spins[N][N], int spins_next[N][N],
double T)
{
    for (int trial = 0; trial < N * N; ++trial)
    {
        // Elegimos un espín aleatorio que no esté en los bordes
        fijos
        int n = 1 + rand() % (N - 2); // solo posiciones
        interiores en X
        int m = rand() % N;

        // Elegimos un vecino aleatorio válido que no esté en los
        bordes fijos
        int candidates[4][2];
        int count = 0;

        // Vecino derecha, solo si no es borde fijo
        if (n + 1 != N - 1)
        {
            candidates[count][0] = n + 1;
            candidates[count][1] = m;
            count++;
        }

        // Vecino izquierda, solo si no es borde fijo
        if (n - 1 != 0)
        {
            candidates[count][0] = n - 1;
            candidates[count][1] = m;
            count++;
        }

        // Vecino arriba
        candidates[count][0] = n;
        candidates[count][1] = (m + 1) % N;
        count++;

        // Vecino abajo
```



```

        candidates[count][0] = n;
        candidates[count][1] = (m - 1 + N) % N;
        count++;

        int choice = rand() % count;
        int n2 = candidates[choice][0];
        int m2 = candidates[choice][1];

        // Calculamos el cambio de energía localmente (O(1)) y
        decidimos aceptación
        int dE = delta_energy_swap(spins, n, m, n2, m2);
        double p = min(1.0, exp(-dE / T));
        if (random_double() < p)
        {
            int temp = spins[n][m];
            spins[n][m] = spins[n2][m2];
            spins[n2][m2] = temp;
            spins_next[n][m] = spins[n][m];
            spins_next[n2][m2] = spins[n2][m2];
        }
        else
        {
            spins_next[n][m] = spins[n][m];
            spins_next[n2][m2] = spins[n2][m2];
        }
    }
}

// Calcula y almacena las curvas de magnetización para diferentes
temperaturas
void calculate_magnetization()
{
    #define N_MAG 128
    constexpr int N_TEMPS = 10;
    const double T_m[N_TEMPS] = {1.5, 1.7, 1.9, 2.1, 2.3, 2.5,
2.7, 2.9, 3.1, 3.3};
    int i, j, k, n, m;

    // === CAMBIO 1: Eliminamos E, E2, M2 globales de aquí. ===
    // Por qué: Deben inicializarse por separado para cada
    repetición y tamaño,
    // así que usaremos vectores dentro del bucle como
    propusiste.

    FILE *mag32 = fopen("magn32.txt", "w");
    FILE *mag64 = fopen("magn64.txt", "w");
    FILE *mag128 = fopen("magn128.txt", "w");
    FILE *prof32 = fopen("perfil32.txt", "w");
    FILE *prof64 = fopen("perfil64.txt", "w");
    FILE *prof128 = fopen("perfil128.txt", "w");
    FILE *files_prof[3] = {prof32, prof64, prof128};

    if (!mag32 || !mag64 || !mag128)
    {
        std::cerr << "Error abriendo archivos de magnetización."
<< std::endl;

```

```

        return;
    }

    // Número de experimentos independientes por temperatura para
    // sacar estadísticas
    const int N_experimentos = 10;

    int sizes[3] = {32, 64, 128};
    FILE *files[3] = {mag32, mag64, mag128};

    // Hacemos los cálculos de la magnetización
    for (i = 0; i < N_TEMPS; i++)
    {
        for (int s_idx = 0; s_idx < 3; ++s_idx)
        {
            int L = sizes[s_idx];

            // === CAMBIO 3: Adaptamos los vectores para E, E2 y
M2 ===
            // Por qué: Para almacenar el promedio de E, E2 y M2
            // de CADA repetición independiente.
            std::vector<double> medidas_M(N_experimentos);
            std::vector<double> medidas_E(N_experimentos);
            std::vector<double> medidas_E2(N_experimentos);
            std::vector<double> medidas_M2(N_experimentos);

            // Repetimos el experimento N veces para calcular los
            // errores en paralelo
            #pragma omp parallel for schedule(dynamic)
            for (int rep = 0; rep < N_experimentos; ++rep)
            {
                unsigned int seed = (unsigned int)time(NULL) ^
(unsigned int)rep ^ (unsigned int)omp_get_thread_num();
                int sN_local[N_MAG][N_MAG];
                for (int nn = 0; nn < L; ++nn)
                    for (int mm = 0; mm < L; ++mm)
                        sN_local[nn][mm] = -1;

                int half_spins = (L * L) / 2;
                int count_ones = 0;
                while (count_ones < half_spins)
                {
                    int rn = rand_r(&seed) % L;
                    int rm = rand_r(&seed) % L;
                    if (sN_local[rn][rm] == -1)
                    {
                        sN_local[rn][rm] = 1;
                        count_ones++;
                    }
                }

                int sN_next[N_MAG][N_MAG];
                for (int nn = 0; nn < L; ++nn)
                    for (int mm = 0; mm < L; ++mm)
                        sN_next[nn][mm] = sN_local[nn][mm];
            }
        }
    }

```

```

const int total_mc_trials = 1000000;
int mc_steps = total_mc_trials / (L * L);
int extra_trials = total_mc_trials % (L * L);

double suma_E = 0.0;
double suma_E2 = 0.0;
int numero_medidas = 0;

double energia_actual = total_energy_L((const int
(*)[128])sN_local, L);

for (int step = 0; step < mc_steps; ++step)
{
    for (int trial = 0; trial < L * L; ++trial)
    {
        int n = 1 + rand_r(&seed) % (L - 2);
        int m = rand_r(&seed) % L;

        int n2, m2;
        select_neighbor_bc(n, m, L, n2, m2);

        int dE = delta_energy_swap_L((const int
(*)[128])sN_local, n, m, n2, m2, L);
        double p = min(1.0, exp(-dE / T_m[i]));
        if (random_double_r(seed) < p)
        {
            int temp = sN_local[n][m];
            sN_local[n][m] = sN_local[n2][m2];
            sN_local[n2][m2] = temp;
            sN_next[n][m] = sN_local[n][m];
            sN_next[n2][m2] = sN_local[n2][m2];
            energia_actual += dE;
        }
    }
    suma_E += energia_actual;
    suma_E2 += (energia_actual * energia_actual);
    numero_medidas++;
}

if (extra_trials > 0)
{
    for (j = 0; j < extra_trials; ++j)
    {
        int n = 1 + rand_r(&seed) % (L - 2);
        int m = rand_r(&seed) % L;

        int n2, m2;
        select_neighbor_bc(n, m, L, n2, m2);

        int dE = delta_energy_swap_L((const int
(*)[128])sN_local, n, m, n2, m2, L);
        double p = min(1.0, exp(-dE / T_m[i]));
        if (random_double_r(seed) < p)
        {
            int temp = sN_local[n][m];
            sN_local[n][m] = sN_local[n2][m2];

```

```

        sN_local[n2][m2] = temp;
        sN_next[n][m] = sN_local[n][m];
        sN_next[n2][m2] = sN_local[n2][m2];
        energia_actual += dE;
    }
}
suma_E += energia_actual;
suma_E2 += (energia_actual * energia_actual);
numero_medidas++;
}

double m_top = 0.0;
double m_bottom = 0.0;
for (j = 0; j < L; ++j)
{
    for (k = 0; k < L; ++k)
    {
        if (k < L / 2)
            m_top += sN_local[j][k];
        else
            m_bottom += sN_local[j][k];
    }
}

double particles_per_half = (L * L) / 2.0;
double mag_domain = (fabs(m_top) /
particles_per_half + fabs(m_bottom) / particles_per_half) / 2.0;
double num_particulas = L * L;

medidas_M[rep] = mag_domain;
medidas_M2[rep] = (mag_domain * mag_domain);
medidas_E[rep] = (suma_E / numero_medidas) /
num_particulas;
medidas_E2[rep] = (suma_E2 / numero_medidas) /
(num_particulas * num_particulas);

// Solo el hilo 0 realiza la escritura del perfil
para evitar colisiones en IO
if (rep == N_experimentos - 1)
{
    #pragma omp critical
    {
        export_density_profile(files_prof[s_idx],
(const int (*)(N_MAG))sN_local, L, T_m[i]);
    }
}

// === CAMBIO 6: Calculamos medias y errores para
todas las magnitudes termodinámicas ===
double media_M, err_M;
calcular_error_montecarlo(medidas_M, media_M, err_M);

double media_M2, err_M2;
calcular_error_montecarlo(medidas_M2, media_M2,
err_M2);

```

```

        double media_E, err_E;
        calcular_error_montecarlo(medidas_E, media_E, err_E);

        double media_E2, err_E2;
        calcular_error_montecarlo(medidas_E2, media_E2,
err_E2);

        // === CAMBIO 1.3: Cálculo de calor específico y
susceptibilidad ===
        // Como las medias que hemos obtenido están "por
partícula" ( $\langle e \rangle$ ,  $\langle e^2 \rangle$ ,  $\langle m \rangle$ ,  $\langle m^2 \rangle$ ),
        // aplicamos el factor multiplicativo ( $L \cdot L$ ) derivado
de adaptar la fórmula extensiva del PDF a valores intensivos.
        double num_particulas = L * L;
        double T = T_m[i];

        double c_N = (num_particulas / (T * T)) * (media_E2 -
(media_E * media_E));
        double chi_N = (num_particulas / T) * (media_M2 -
(media_M * media_M));

        // === CAMBIO 1.3 (Parte 2): Añadidas c_N y chi_N al
archivo de salida ===
        // Formato CSV por tabulaciones: T |  $\langle M \rangle$  | err_M |
 $\langle E \rangle$  | err_E | c_N | chi_N
        fprintf(files[s_idx],
"%lf\t%e\t%e\t%e\t%e\t%e\t%e\n", T_m[i], media_M, err_M, media_E,
err_E, c_N, chi_N);
    }
}

fclose(mag32);
fclose(mag64);
fclose(mag128);
}

// -----
// NUEVA FUNCIÓN: Conteo global de espines (+1 y -1)
// -----
void export_global_spin_count(FILE* file, const int spins[128]
[128], int L, double T)
{
    int count_plus = 0;
    int count_minus = 0;

    for (int i = 0; i < L; ++i)
    {
        for (int j = 0; j < L; ++j)
        {
            if (spins[i][j] == 1)
            {
                count_plus++;
            }
            else if (spins[i][j] == -1)
            {

```

```

        count_minus++;
    }
}

// Formato CSV por tabulaciones: Temperatura | Cantidad +1 |
// Cantidad -1
fprintf(file, "%lf\t%d\t%d\n", T, count_plus, count_minus);
}

// -----
// NUEVA FUNCIÓN: Perfil de densidad fila por fila
// -----
void export_density_profile(FILE* file, const int spins[N][N],
int L, double T)
{
    // Iteramos sobre la coordenada Y (que en tu matriz es el
    // segundo índice, 'm')
    for (int m = 0; m < L; ++m)
    {
        int count_plus = 0;
        int count_minus = 0;

        // Sumamos todos los valores a lo largo de la coordenada
        // X (primer índice, 'n')
        for (int n = 0; n < L; ++n)
        {
            if (spins[n][m] == 1)
                count_plus++;
            else if (spins[n][m] == -1)
                count_minus++;
        }

        // Exportamos: Temperatura, Coordenada Y, Cantidad +1,
        // Cantidad -1
        fprintf(file, "%lf\t%d\t%d\t%d\n", T, m, count_plus,
count_minus);
    }
    // Dejamos una línea en blanco al terminar la matriz.
    // Esto es muy útil en Gnuplot para separar los datos de
    // distintas temperaturas.
    fprintf(file, "\n");
}

//
// -----
// NUEVA FUNCIÓN: Calcula magnetización para m0 distinto de cero
//
// -----
void calculate_magnetization_m0(double m0)
{
    #define N_MAG 128
    constexpr int N_TEMPS = 10;
    const double T_m[N_TEMPS] = {1.5, 1.7, 1.9, 2.1, 2.3, 2.5,
2.7, 2.9, 3.1, 3.3};
    int i; // Mantenemos solo 'i' como global para el bucle

```

exterior de temperaturas

// Abrimos archivos con sufijo _m0 para no sobrescribir los datos de m0=0

```
FILE *mag32 = fopen("magn32_m0.txt", "w");
FILE *mag64 = fopen("magn64_m0.txt", "w");
FILE *mag128 = fopen("magn128_m0.txt", "w");
FILE *prof32 = fopen("perfil32_m0.txt", "w");
FILE *prof64 = fopen("perfil64_m0.txt", "w");
FILE *prof128 = fopen("perfil128_m0.txt", "w");
```

```
FILE *files[3] = {mag32, mag64, mag128};
FILE *files_prof[3] = {prof32, prof64, prof128};
```

```
if (!mag32 || !mag64 || !mag128)
{
    std::cerr << "Error abriendo archivos de magnetización
m0." << std::endl;
    return;
}
```

```
const int N_experimentos = 10;
int sizes[3] = {32, 64, 128};
```

// Calculamos la fracción teórica x

```
double x_frac = (1.0 + m0) / 2.0;
```

```
for (i = 0; i < N_TEMPS; i++)
{
```

```
    for (int s_idx = 0; s_idx < 3; ++s_idx)
    {
```

```
        int L = sizes[s_idx];
        int L_split = static_cast<int>(round(x_frac * L));
```

```
        std::vector<double> medidas_M(N_experimentos);
        std::vector<double> medidas_E(N_experimentos);
        std::vector<double> medidas_E2(N_experimentos);
        std::vector<double> medidas_M2(N_experimentos);
```

// === CAMBIO OpenMP: Paralelizamos el bucle de repeticiones de Montecarlo ===

```
#pragma omp parallel for schedule(dynamic)
for (int rep = 0; rep < N_experimentos; ++rep)
{
```

// Generamos una semilla única por hilo y repetición para evitar colisiones RNG

```
    unsigned int seed = (unsigned int)time(NULL) ^
(unsigned int)rep ^ (unsigned int)omp_get_thread_num();
```

// === CAMBIO OpenMP: Matriz sN pasa a ser sN_local (privada para cada hilo) ===

```
    int sN_local[N_MAG][N_MAG];
```

// Inicializamos la matriz local a -1

```
    for (int nn = 0; nn < L; ++nn)
        for (int mm = 0; mm < L; ++mm)
```

```

        sN_local[nn][mm] = -1;

        // === CAMBIO OpenMP: Inicialización 'm0' en
        línea (Thread-safe) ===
        // En lugar de llamar a initialize_kawasaki_m0()
        que usa el rand() global,
        // generamos los espines aquí dentro usando
        rand_r(&seed)

        int total_spins = L * L;
        int count_plus_needed =
static_cast<int>(round(x_frac * total_spins));
        int count_ones = 0;
        while (count_ones < count_plus_needed)
        {
            int rn = rand_r(&seed) % L;
            int rm = rand_r(&seed) % L;
            if (sN_local[rn][rm] == -1)
            {
                sN_local[rn][rm] = 1;
                count_ones++;
            }
        }

        // Matriz para guardar el estado siguiente
        int sN_next[N_MAG][N_MAG];
        for (int nn = 0; nn < L; ++nn)
            for (int mm = 0; mm < L; ++mm)
                sN_next[nn][mm] = sN_local[nn][mm];

        const int total_mc_trials = 1000000;
        int mc_steps = total_mc_trials / (L * L);
        int extra_trials = total_mc_trials % (L * L);

        // Control para exportar el fotograma (sólo hilo
        de la repetición 0 lo hará)
        bool capture_evolution = (s_idx == 2 && rep ==
0);

        FILE *f_evol = nullptr;

        if (capture_evolution)
        {
            char filename_evol[100];
            snprintf(filename_evol,
sizeof(filename_evol), "ising_evol_m0_%.2f_T_%.2f.dat", m0,
T_m[i]);

            f_evol = fopen(filename_evol, "w");
        }

        int writes_evol = 0;
        int max_writes_evol = std::max(1, mc_steps / 10);
        int energy_threshold_evol = L * 2;
        double last_energy_evol = 0;

        double suma_E = 0.0;
        double suma_E2 = 0.0;
        int numero_medidas = 0;

```



```

        // Evaluamos energía inicial (pasando el puntero
local)
        double energia_actual = total_energy_L((const int
(*)[128])sN_local, L);

        if (capture_evolution && f_evol)
        {
            write_lattice(f_evol, (const int (*)
[N])sN_local);
            last_energy_evol = energia_actual;
        }

        for (int step = 0; step < mc_steps; ++step)
        {
            for (int trial = 0; trial < L * L; ++trial)
            {
                // === CAMBIO OpenMP: Usamos rand_r en
todo el Montecarlo ===
                int n = 1 + rand_r(&seed) % (L - 2);
                int m = rand_r(&seed) % L;

                int n2, m2;
                // OJO: select_neighbor_bc por dentro
sigue usando rand(). Si te da problemas de rendimiento
                // lo ideal es no usarla y poner el
vecino directamente aquí dentro como en metropolis_sweep.
                // La mantengo para respetar tu
estructura exacta.
                select_neighbor_bc(n, m, L, n2, m2);

                int dE = delta_energy_swap_L((const int
(*)[128])sN_local, n, m, n2, m2, L);
                double p = min(1.0, exp(-dE / T_m[i]));

                // === CAMBIO OpenMP: random_double pasa
a random_double_r ===
                if (random_double_r(seed) < p)
                {
                    int temp = sN_local[n][m];
                    sN_local[n][m] = sN_local[n2][m2];
                    sN_local[n2][m2] = temp;
                    sN_next[n][m] = sN_local[n][m];
                    sN_next[n2][m2] = sN_local[n2][m2];

                    energia_actual += dE;
                }
            }

            if (capture_evolution && f_evol &&
writes_evol < max_writes_evol)
            {
                if (std::abs(energia_actual -
last_energy_evol) >= energy_threshold_evol)
                {
                    write_lattice(f_evol, (const int (*)

```

```

[N])sN_local);

        last_energy_evol = energia_actual;
        writes_evol++;
    }
}

suma_E += energia_actual;
suma_E2 += (energia_actual * energia_actual);
numero_medidas++;
}

if (extra_trials > 0)
{
    // === CAMBIO OpenMP: Variable de bucle local
    'j_ext' para no chocar con otros hilos ===
    for (int j_ext = 0; j_ext < extra_trials; +
+j_ext)
    {
        int n = 1 + rand_r(&seed) % (L - 2);
        int m = rand_r(&seed) % L;

        int n2, m2;
        select_neighbor_bc(n, m, L, n2, m2);

        int dE = delta_energy_swap_L((const int
(*)[128])sN_local, n, m, n2, m2, L);
        double p = min(1.0, exp(-dE / T_m[i]));
        if (random_double_r(seed) < p)
        {
            int temp = sN_local[n][m];
            sN_local[n][m] = sN_local[n2][m2];
            sN_local[n2][m2] = temp;
            sN_next[n][m] = sN_local[n][m];
            sN_next[n2][m2] = sN_local[n2][m2];

            energia_actual += dE;
        }
    }

    if (capture_evolution && f_evol &&
writes_evol < max_writes_evol)
    {
        if (std::abs(energia_actual -
last_energy_evol) >= energy_threshold_evol)
        {
            write_lattice(f_evol, (const int (*)
[N])sN_local);

            last_energy_evol = energia_actual;
            writes_evol++;
        }
    }

    suma_E += energia_actual;
    suma_E2 += (energia_actual * energia_actual);
    numero_medidas++;
}

```

```

        if (capture_evolution && f_evol)
        {
            fclose(f_evol);
        }

        double m_domain1 = 0.0;
        double m_domain2 = 0.0;

        // === CAMBIO OpenMP: Variables de bucle jj y kk
locales ===
        for (int jj = 0; jj < L; ++jj)
        {
            for (int kk = 0; kk < L; ++kk)
            {
                if (kk < L_split)
                    m_domain1 += sN_local[jj][kk];
                else
                    m_domain2 += sN_local[jj][kk];
            }
        }

        double particles_1 = L_split * L;
        double particles_2 = (L - L_split) * L;

        double mag_1 = (particles_1 > 0) ?
(fabs(m_domain1) / particles_1) : 0.0;
        double mag_2 = (particles_2 > 0) ?
(fabs(m_domain2) / particles_2) : 0.0;

        double mag_domain;
        if (particles_1 == 0) mag_domain = mag_2;
        else if (particles_2 == 0) mag_domain = mag_1;
        else mag_domain = (mag_1 + mag_2) / 2.0;

        double num_particulas = L * L;

        medidas_M[rep] = mag_domain;
        medidas_M2[rep] = (mag_domain * mag_domain);
        medidas_E[rep] = (suma_E / numero_medidas) /
num_particulas;
        medidas_E2[rep] = (suma_E2 / numero_medidas) /
(num_particulas * num_particulas);

        if (rep == N_experimentos - 1)
        {
            // === CAMBIO OpenMP: Impedimos colisión en
escritura a archivo ===
            #pragma omp critical
            {
                export_density_profile(files_prof[s_idx],
(const int (*)(N_MAG))sN_local, L, T_m[i]);
            }
        }
    }
}

```

```

        double media_M, err_M;
        calcular_error_montecarlo(medidas_M, media_M, err_M);

        double media_M2, err_M2;
        calcular_error_montecarlo(medidas_M2, media_M2,
err_M2);

        double media_E, err_E;
        calcular_error_montecarlo(medidas_E, media_E, err_E);

        double media_E2, err_E2;
        calcular_error_montecarlo(medidas_E2, media_E2,
err_E2);

        double num_particulas = L * L;
        double T = T_m[i];

        double c_N = (num_particulas / (T * T)) * (media_E2 -
(media_E * media_E));
        double chi_N = (num_particulas / T) * (media_M2 -
(media_M * media_M));

        fprintf(files[s_idx],
"%lf\t%e\t%e\t%e\t%e\t%e\t%e\n", T_m[i], media_M, err_M, media_E,
err_E, c_N, chi_N);
    }

    fclose(mag32); fclose(mag64); fclose(mag128);
    fclose(prof32); fclose(prof64); fclose(prof128);
}

int main()
{
    auto inicio = chrono::high_resolution_clock::now();
    int semilla = 310809;
    srand(semilla);

    std::cout << "Iniciando simulacion del modelo de Ising
(Dinamica de Kawasaki)..." << std::endl;

    //
=====
    // TAREA 1: Simular la dinámica y obtener fotogramas para
diferentes temperaturas
    //
=====

    std::cout << "\n[1/3] Realizando Tarea 1: Generando
fotogramas de la red..." << std::endl;

    double T_low = 1.0;
    double T_high = 3.5;

    int mc_steps = TOTAL_TRIALS / (N * N);
    int extra_trials = TOTAL_TRIALS % (N * N);
    int max_writes = std::max(1, mc_steps / 10);

```

```

    const int energy_threshold = N * 2;

    //
    -----
    // CASO A: T_baja y T_alta con CONDICIONES PERIÓDICAS EN
    AMBOS EJES
    //
    -----

    std::cout << "        -> Simulando con Condiciones
Periodicas..." << std::endl;

    FILE *f_per_ord_low = fopen("ising_per_ord_low.dat", "w");
    FILE *f_per_des_low = fopen("ising_per_des_low.dat", "w");
    FILE *f_per_ord_high = fopen("ising_per_ord_high.dat", "w");
    FILE *f_per_des_high = fopen("ising_per_des_high.dat", "w");

    if (!f_per_ord_low || !f_per_des_low || !f_per_ord_high || !
f_per_des_high) {
        std::cerr << "Error abriendo archivos para condiciones
periodicas." << std::endl; return 1;
    }

    // Inicializamos: Ordenada para T_low/T_high, Desordenada
para T_low/T_high
    initialize_ordered(sord);
    initialize_disordered(sdesord);

    // Copiamos a las matrices "next"
    for (int n = 0; n < N; ++n) {
        for (int m = 0; m < N; ++m) {
            sord_next[n][m] = sord[n][m];
            sdesord_next[n][m] = sdesord[n][m];
        }
    }

    int last_E_ord = total_energy(sord);
    int last_E_des = total_energy(sdesord);
    int writes_ord = 0, writes_des = 0;

    // Bucle Montecarlo (Usando metropolis_sweep normal) para
T_baja
    for (int step = 0; step < mc_steps; ++step) {
        metropolis_sweep(sord, sord_next, T_low);
        metropolis_sweep(sdesord, sdesord_next, T_low);

        if (writes_ord < max_writes &&
std::abs(total_energy(sord) - last_E_ord) >= energy_threshold) {
            write_lattice(f_per_ord_low, sord);
            last_E_ord = total_energy(sord); writes_ord++;
        }
        if (writes_des < max_writes &&
std::abs(total_energy(sdesord) - last_E_des) >= energy_threshold)
        {
            write_lattice(f_per_des_low, sdesord);
            last_E_des = total_energy(sdesord); writes_des++;
        }
    }

```

```

    }

    // Reiniciamos matrices para probar T_alta con Periódicas
    initialize_ordered(sord);
    initialize_disordered(sdesord);
    for (int n = 0; n < N; ++n) {
        for (int m = 0; m < N; ++m) {
            sord_next[n][m] = sord[n][m];
            sdesord_next[n][m] = sdesord[n][m];
        }
    }

    last_E_ord = total_energy(sord); last_E_des =
total_energy(sdesord);
    writes_ord = 0; writes_des = 0;

    // Bucle Montecarlo (Usando metropolis_sweep normal) para
T_alta
    for (int step = 0; step < mc_steps; ++step) {
        metropolis_sweep(sord, sord_next, T_high);
        metropolis_sweep(sdesord, sdesord_next, T_high);

        if (writes_ord < max_writes &&
std::abs(total_energy(sord) - last_E_ord) >= energy_threshold) {
            write_lattice(f_per_ord_high, sord);
            last_E_ord = total_energy(sord); writes_ord++;
        }
        if (writes_des < max_writes &&
std::abs(total_energy(sdesord) - last_E_des) >= energy_threshold)
{
            write_lattice(f_per_des_high, sdesord);
            last_E_des = total_energy(sdesord); writes_des++;
        }
    }

    fclose(f_per_ord_low); fclose(f_per_des_low);
    fclose(f_per_ord_high); fclose(f_per_des_high);

    //
-----
    // CASO B: T_baja y T_alta con BORDES FIJOS EN X (Para
observar mejor los dominios)
    //
-----

    std::cout << "          -> Simulando con Bordes Fijos (eje X)..."
<< std::endl;

    FILE *f_bc_ord_low = fopen("ising_bc_ord_low.dat", "w");
    FILE *f_bc_des_low = fopen("ising_bc_des_low.dat", "w");
    FILE *f_bc_ord_high = fopen("ising_bc_ord_high.dat", "w");
    FILE *f_bc_des_high = fopen("ising_bc_des_high.dat", "w");

    if (!f_bc_ord_low || !f_bc_des_low || !f_bc_ord_high || !
f_bc_des_high) {
        std::cerr << "Error abriendo archivos para bordes fijos."

```

```

<< std::endl; return 1;
}

// Inicializamos usando la función con bordes fijos
initialize_ordered(sord);
initialize_disordered_bc(sdesord);
for (int n = 0; n < N; ++n) {
    for (int m = 0; m < N; ++m) {
        sord_next[n][m] = sord[n][m];
        sdesord_next[n][m] = sdesord[n][m];
    }
}

last_E_ord = total_energy(sord); last_E_des =
total_energy(sdesord);
writes_ord = 0; writes_des = 0;

// Bucle Montecarlo (Usando metropolis_sweep_bc) para T_baja
for (int step = 0; step < mc_steps; ++step) {
    metropolis_sweep_bc(sord, sord_next, T_low);
    metropolis_sweep_bc(sdesord, sdesord_next, T_low);

    if (writes_ord < max_writes &&
std::abs(total_energy(sord) - last_E_ord) >= energy_threshold) {
        write_lattice(f_bc_ord_low, sord);
        last_E_ord = total_energy(sord); writes_ord++;
    }
    if (writes_des < max_writes &&
std::abs(total_energy(sdesord) - last_E_des) >= energy_threshold)
{
        write_lattice(f_bc_des_low, sdesord);
        last_E_des = total_energy(sdesord); writes_des++;
    }
}

// Reiniciamos matrices para probar T_alta con bordes fijos
initialize_ordered(sord);
initialize_disordered_bc(sdesord);
for (int n = 0; n < N; ++n) {
    for (int m = 0; m < N; ++m) {
        sord_next[n][m] = sord[n][m];
        sdesord_next[n][m] = sdesord[n][m];
    }
}

last_E_ord = total_energy(sord); last_E_des =
total_energy(sdesord);
writes_ord = 0; writes_des = 0;

// Bucle Montecarlo (Usando metropolis_sweep_bc) para T_alta
for (int step = 0; step < mc_steps; ++step) {
    metropolis_sweep_bc(sord, sord_next, T_high);
    metropolis_sweep_bc(sdesord, sdesord_next, T_high);

    if (writes_ord < max_writes &&
std::abs(total_energy(sord) - last_E_ord) >= energy_threshold) {

```

```

        write_lattice(f_bc_ord_high, sord);
        last_E_ord = total_energy(sord); writes_ord++;
    }
    if (writes_des < max_writes &&
std::abs(total_energy(sdesord) - last_E_des) >= energy_threshold)
{
    write_lattice(f_bc_des_high, sdesord);
    last_E_des = total_energy(sdesord); writes_des++;
}
}

fclose(f_bc_ord_low); fclose(f_bc_des_low);
fclose(f_bc_ord_high); fclose(f_bc_des_high);

//
=====
// TAREAS 2 a 7: Cálculos termodinámicos con magnetización
nula ( $m_0 = 0$ )
//
=====
// Esta función encapsula todo: curvas de magnetización,
energía, calor específico,
// susceptibilidad y los perfiles de densidad en el eje Y.
std::cout << "\n[2/3] Realizando Tareas 2-7: Termodinamica
( $m_0 = 0$ )..." << std::endl;
calculate_magnetization();

//
=====
// TAREA 8: Cálculos termodinámicos partiendo de una
magnetización NO nula ( $m_0 \neq 0$ )
//
=====
// Ejecutamos exactamente la misma termodinámica
(magnetización, E, c_N, chi_N, perfiles)
// pero inicializando el sistema con la fracción asimétrica
correspondiente a  $m_0$ .
std::cout << "\n[3/3] Realizando Tarea 8: Termodinamica ( $m_0 \neq 0$ )..." << std::endl;
double m0_deseada_1 = 0.5; // Probamos con un  $m_0$  del 50%
double m0_deseada_2 = 0.8; // Probamos con un  $m_0$  del 80%
calculate_magnetization_m0(m0_deseada_1);
calculate_magnetization_m0(m0_deseada_2);

std::cout << "\nSimulacion finalizada con exito. Revisa los
archivos de salida generados." << std::endl;

auto fin = chrono::high_resolution_clock::now();
chrono::duration<double, milli> tiempo_ejecucion = fin -
inicio;
cout << "El código multi-run completo tardó: " <<
tiempo_ejecucion.count() << " milisegundos." << endl;
return 0;
}

```


Asimismo, se utilizaron scripts de python en pos de analizar los resultados dados en la simulación.

```
In [ ]: # =====
# ANIMACION ISING
#
# Genera una animación a partir de un fichero de datos con la configuraci
# del retículo en cada instante de tiempo
#
# El fichero debe estructurarse de la siguiente forma:
#
#   s(1,1)_1, s(1,2)_1, ..., s(1,M)_1
#   s(2,1)_1, s(2,2)_1, ..., s(2,M)_1
#   (...)
#   s(N,1)_1, s(N,2)_1, ..., s(N,M)_1
#
#   s(1,1)_2, s(1,2)_2, ..., s(1,M)_2
#   s(2,1)_2, s(2,2)_2, ..., s(2,M)_2
#   (...)
#   s(N,1)_2, s(N,2)_2, ..., s(N,M)_2
#
#   s(1,1)_3, s(1,2)_3, ..., s(1,M)_3
#   s(2,1)_3, s(2,2)_3, ..., s(2,M)_3
#   (...)
#   s(N,1)_3, s(N,2)_3, ..., s(N,M)_3
#
#   (...)
#
# donde s(i,j)_k es el valor del spin en la fila i-ésima y la columna
# j-ésima en el instante k. M es el número de columnas y N el número
# de filas en el retículo. Los valores del spin deben ser +1 ó -1.
# El programa asume que las dimensiones del retículo no cambian a lo
# largo del tiempo.
#
# Si solo se especifica un instante de tiempo, se genera una imagen en pd
# en lugar de una animación
#
# Se puede configurar la animación cambiando el valor de las variables
# de la sección "Parámetros"
#
# =====

# Importa los módulos necesarios
from matplotlib import pyplot as plt
from matplotlib.animation import FuncAnimation
import numpy as np
import io

# Parámetros
# =====
file_in = "ising_evol_m0_0.50_T_3.10.dat" # Nombre del fichero de datos
file_out = "fotogram" # Nombre del fichero de salida (sin extensión)
interval = 100 # Tiempo entre fotogramas en milisegundos
save_to_file = True # False: muestra la animación por pantalla,
                    # True: la guarda en un fichero
dpi = 150 # Calidad del vídeo de salida (dots per inch)
```

```

# Lectura del fichero de datos
# =====
# Lee el fichero a una cadena de texto
with open(file_in, "r") as f:
    data_str = f.read()

# Inicializa la lista con los datos de cada fotograma.
# frames_data[j] contiene los datos del fotograma j-ésimo
frames_data = list()

# Itera sobre los bloques de texto separados por líneas vacías
# (cada bloque corresponde a un instante de tiempo)
for frame_data_str in data_str.split("\n\n"):

    # Ignora los bloques vacíos (como saltos de línea extra al final del
    if not frame_data_str.strip():
        continue

    # Almacena el bloque en una matriz
    # (io.StringIO permite leer una cadena de texto como si fuera un
    # fichero, lo que nos permite usar la función loadtxt de numpy)
    frame_data = np.loadtxt(io.StringIO(frame_data_str), delimiter=",")

    # Añade los datos del fotograma (la configuración del sistema)
    # a la lista
    frames_data.append(frame_data)

# Creación de la animación/gráfico
# =====
# Crea los objetos figure y axis
fig, ax = plt.subplots()

# Define el rango de los ejes
ax.axis("off") # No muestra los ejes

# Representa el primer fotograma
im = ax.imshow(frames_data[0], cmap="binary", vmin=-1, vmax=+1)

# Función que actualiza la configuración del sistema en la animación
def update(j_frame, frames_data, im):
    # Actualiza el gráfico con la configuración del sistema
    im.set_data(frames_data[j_frame])

    return im,

# Calcula el nº de fotogramas o instantes de tiempo
nframes = len(frames_data)

# Si hay más de un instante de tiempo, genera la animación
if nframes > 1:
    animation = FuncAnimation(
        fig, update,
        fargs=(frames_data, im), frames=len(frames_data), blit=True,

    # Muestra por pantalla o guarda según parámetros
    if save_to_file:
        animation.save("{} .mp4" .format(file_out), dpi=dpi)
    else:
        plt.show()
# En caso contrario, muestra o guarda una imagen

```

```

else:
    # Muestra por pantalla o guarda según parámetros
    if save_to_file:
        fig.savefig("{}_pdf".format(file_out))
    else:
        plt.show()

```

```

In [ ]: import numpy as np
import matplotlib.pyplot as plt
import os

# =====
# CONFIGURACIÓN DEL SCRIPT
# =====
# Nombres de los archivos a leer. Cámbialos por los que quieras analizar
archivos = ['magn32_m0_0.50.txt', 'magn64_m0_0.50.txt', 'magn128_m0_0.50.
etiquetas = ['N = 32', 'N = 64', 'N = 128']
colores = ['blue', 'orange', 'green']

# Crear la figura (1 solo gráfico grande para ver bien las barras de erro
fig, ax = plt.subplots(figsize=(8, 6))
fig.suptitle('Energía Interna - Modelo de Ising (Dinámica de Kawasaki)',

# =====
# LECTURA Y GRAFICADO DE DATOS
# =====
for archivo, etiqueta, color in zip(archivos, etiquetas, colores):
    if not os.path.exists(archivo):
        print(f"Advertencia: No se encontró el archivo {archivo}. Se omit
        continue

    # Cargar los datos. np.loadtxt separa automáticamente por tabulacione
    # Columnas: 0:T | 1:<M> | 2:err_M | 3:<E> | 4:err_E | 5:c_N | 6:chi_N
    datos = np.loadtxt(archivo)

    T = datos[:, 0]
    E = datos[:, 3]          # Columna de la Energía media
    err_E = datos[:, 4]      # Columna del Error de la Energía media

    # Gráfica de Energía vs Temperatura con barras de error
    # Usamos errorbar en lugar de plot para visualizar la desviación esta
    ax.errorbar(T, E, yerr=err_E, fmt='-o', markersize=5, capsize=4,
                color=color, label=etiqueta, linewidth=1.5, alpha=0.8)

# =====
# FORMATO FINAL Y GUARDADO
# =====
ax.set_xlabel('Temperatura ($J/k_B$)', fontsize=12)
ax.set_ylabel('Energía media por partícula $\langle e \rangle$ ($J$)',
ax.set_title('Evolución de la energía frente a la temperatura', fontsize=
ax.grid(True, linestyle='--', alpha=0.6)

# Añadir la leyenda
ax.legend(fontsize=11)

plt.tight_layout() # Ajusta los márgenes
plt.subplots_adjust(top=0.90) # Deja espacio para el título principal

# Guardar la gráfica en una imagen de alta calidad y mostrarla en pantall
plt.savefig('energia_media_resultados_m0_0.50.png', dpi=300)

```

```
print("Gráfica guardada como 'energia_media_resultados_m0_0.50.png'")
plt.show()
```

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
import os

# =====
# CONFIGURACIÓN DEL SCRIPT
# =====
archivo_perfil = 'perfil128_m0_0.80.txt'
L = 128

# Seleccionamos temperaturas específicas para que el gráfico no esté saturado
temperaturas_a_graficar = [1.5, 2.3, 3.1]

if not os.path.exists(archivo_perfil):
    print(f"Error: No se encontró el archivo {archivo_perfil}")
    exit()

datos = np.loadtxt(archivo_perfil)

# =====
# PROCESAMIENTO Y GRAFICADO (Gráfico de Barras)
# =====
plt.figure(figsize=(10, 6))

temperaturas_unicas = np.unique(datos[:, 0])
# Definimos un ancho para las barras y un desfase (offset) para que no se solapen
width = 0.25
offsets = [-width, 0, width]

for idx, T_target in enumerate(temperaturas_a_graficar):
    if T_target in temperaturas_unicas:
        datos_T = datos[datos[:, 0] == T_target]

        eje_Y = datos_T[:, 1]
        densidad = datos_T[:, 2] / L # Densidad = (nº espines + 1) / L

        # Graficamos barras:
        # eje_Y + offset para que las barras de distintas T queden una al lado de la otra
        plt.bar(eje_Y + offsets[idx], densidad, width=width,
                label=f'T = {T_target} $J/k_B$, alpha=0.7)

# =====
# FORMATO FINAL Y GUARDADO
# =====
plt.title(f'Perfil de Densidad para N = {L}', fontsize=14)
plt.xlabel('Coordenada Y (Posición de la fila)', fontsize=12)
plt.ylabel('Densidad (Fracción de espines + 1)', fontsize=12)

plt.ylim(-0.05, 1.05)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.legend()

plt.tight_layout()
plt.savefig(f'perfil_densidad_{L}.png', dpi=300)
print(f"Gráfica de barras guardada como 'perfil_densidad_barras_{L}.png'")
plt.show()
```

```

In [ ]: import numpy as np
import matplotlib.pyplot as plt
import os

# =====
# CONFIGURACIÓN DEL SCRIPT
# =====
# Nombres de los archivos a leer. Puedes cambiarlos por los terminados en
archivos = ['magn32_m0_0.80.txt', 'magn64_m0_0.80.txt', 'magn128_m0_0.80.
etiquetas = ['N = 32', 'N = 64', 'N = 128']
colores = ['blue', 'orange', 'green']

# Crear la figura con 3 subgráficos (1 fila, 3 columnas)
fig, axs = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Análisis Termodinámico - Modelo de Ising (Dinámica de Kawa

# =====
# LECTURA Y GRAFICADO DE DATOS
# =====
for archivo, etiqueta, color in zip(archivos, etiquetas, colores):
    if not os.path.exists(archivo):
        print(f"Advertencia: No se encontró el archivo {archivo}. Se omit
        continue

    # Cargar los datos. np.loadtxt separa automáticamente por tabulacione
    # Columnas: 0:T | 1:<M> | 2:err_M | 3:<E> | 4:err_E | 5:c_N | 6:chi_N
    datos = np.loadtxt(archivo)

    T = datos[:, 0]
    M = datos[:, 1]
    c_N = datos[:, 5]
    chi_N = datos[:, 6]

    # 1. Gráfica de Magnetización vs Temperatura
    axs[0].plot(T, M, marker='o', markersize=4, linestyle='--', color=colo
    axs[0].set_xlabel('Temperatura ($J/k_B$)')
    axs[0].set_ylabel('Magnetización por partícula $\langle m \rangle$')
    axs[0].set_title('Curva de Magnetización')
    axs[0].grid(True, linestyle='--', alpha=0.6)

    # 2. Gráfica de Calor Específico vs Temperatura
    axs[1].plot(T, c_N, marker='^', markersize=4, linestyle='--', color=co
    axs[1].set_xlabel('Temperatura ($J/k_B$)')
    axs[1].set_ylabel('Calor Específico $c_N$')
    axs[1].set_title('Calor Específico')
    axs[1].grid(True, linestyle='--', alpha=0.6)

    # 3. Gráfica de Susceptibilidad Magnética vs Temperatura
    axs[2].plot(T, chi_N, marker='s', markersize=4, linestyle='--', color=
    axs[2].set_xlabel('Temperatura ($J/k_B$)')
    axs[2].set_ylabel('Susceptibilidad $\chi_N$')
    axs[2].set_title('Susceptibilidad Magnética')
    axs[2].grid(True, linestyle='--', alpha=0.6)

# =====
# FORMATO FINAL Y GUARDADO
# =====
# Poner la leyenda en el primer gráfico
axs[0].legend()

```

```
plt.tight_layout() # Ajusta los márgenes para que no se superpongan los t  
plt.subplots_adjust(top=0.88) # Deja espacio para el título principal  
  
# Guardar la gráfica en una imagen de alta calidad y mostrarla en pantall  
plt.savefig('termodinamica_resultados.png', dpi=300)  
print("Gráfica guardada como 'termodinamica_resultados.png'")  
plt.show()
```

4. Resultados y discusión.

Dado que se realizaron cuatro simulaciones distintas, compararemos resultados entre las simulaciones optimizadas y sin optimizar realizadas tanto en mi PC personal como en el cluster, pero solo se estudiará y comentará a fondo los resultados conseguidos con la simulación optimizada realizada en mi PC.

1. Evolución temporal del modelo. Fotogramas.

1. Magnetización inicial nula.

Se captuaron los fotogramas finales de una animación de la dinámica de Kawasaki para distintas temperaturas, con un tamaño de red de 128x128.

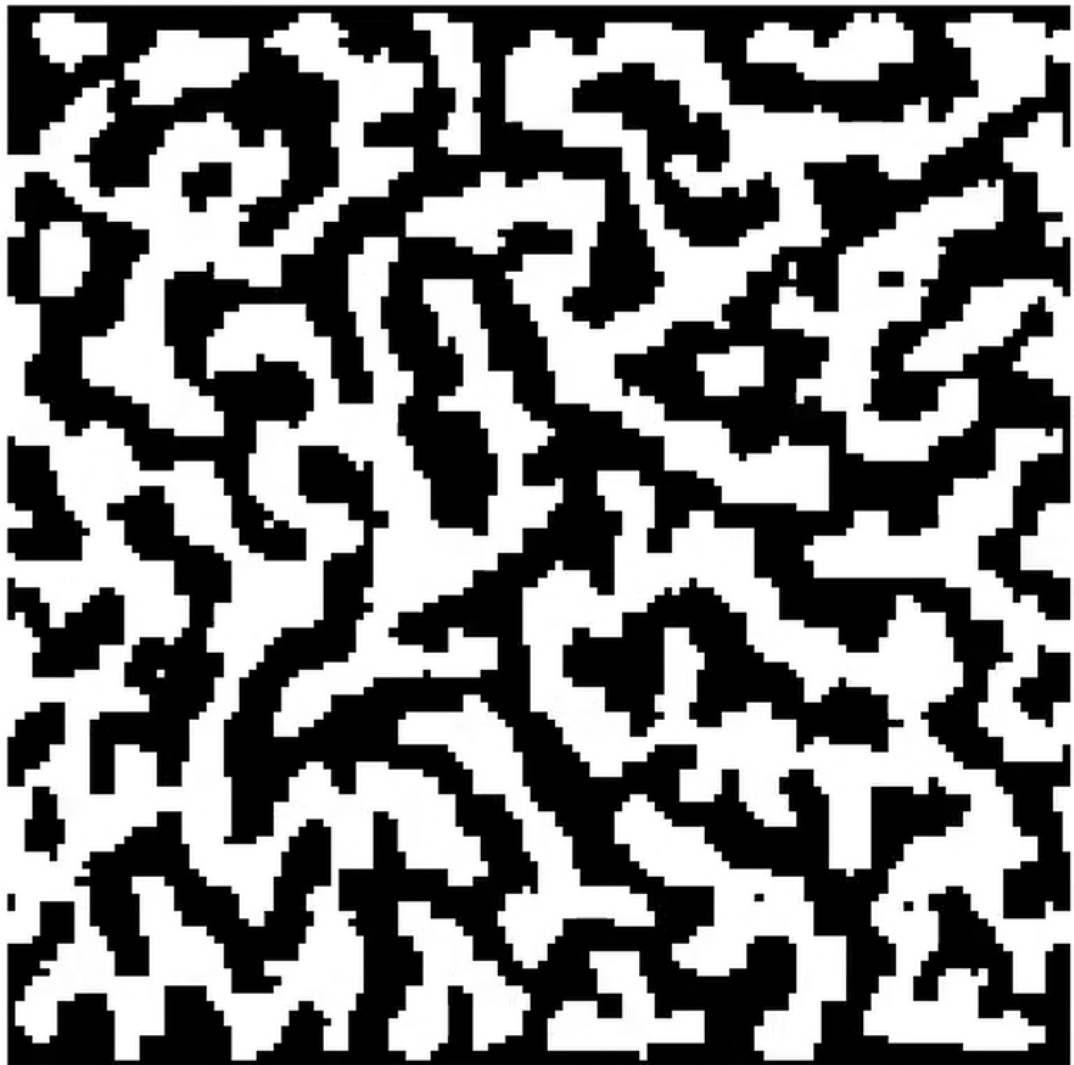


Figura 1: Fotograma de la distribución de espines de la red del modelo de Ising en la

dinámica de Kawasaki con magnetización inicial nula y a una temperatura $T = 1.0J/k_B$, con condiciones de contorno periódicas en el eje X y los espines en los bordes superior e inferior de la red fijos.

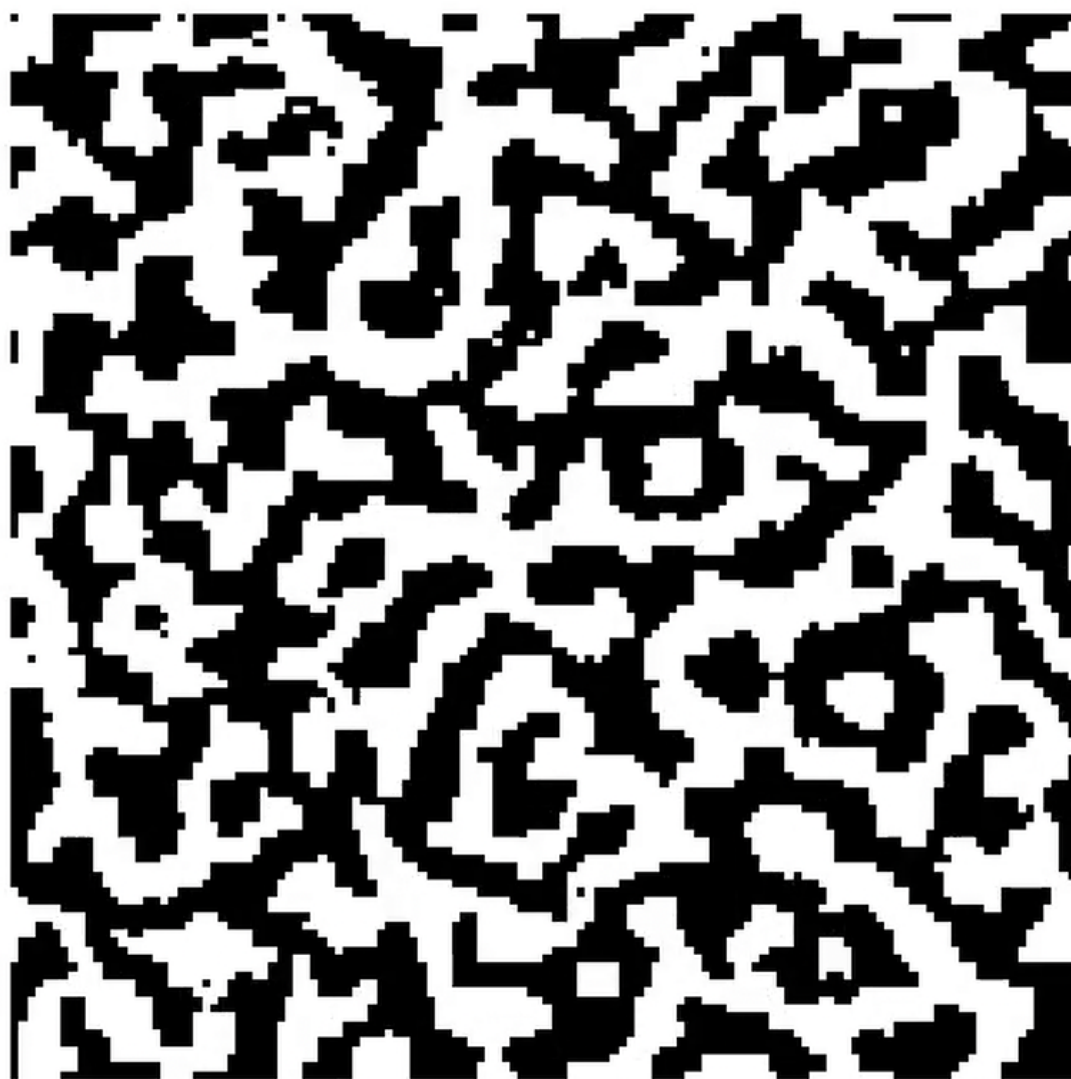


Figura 2: *Fotograma de la distribución de espines de la red del modelo de Ising en la dinámica de Kawasaki con magnetización inicial nula y a una temperatura $T = 1.0J/k_B$ para condiciones de contorno periódicas en todas direcciones.*

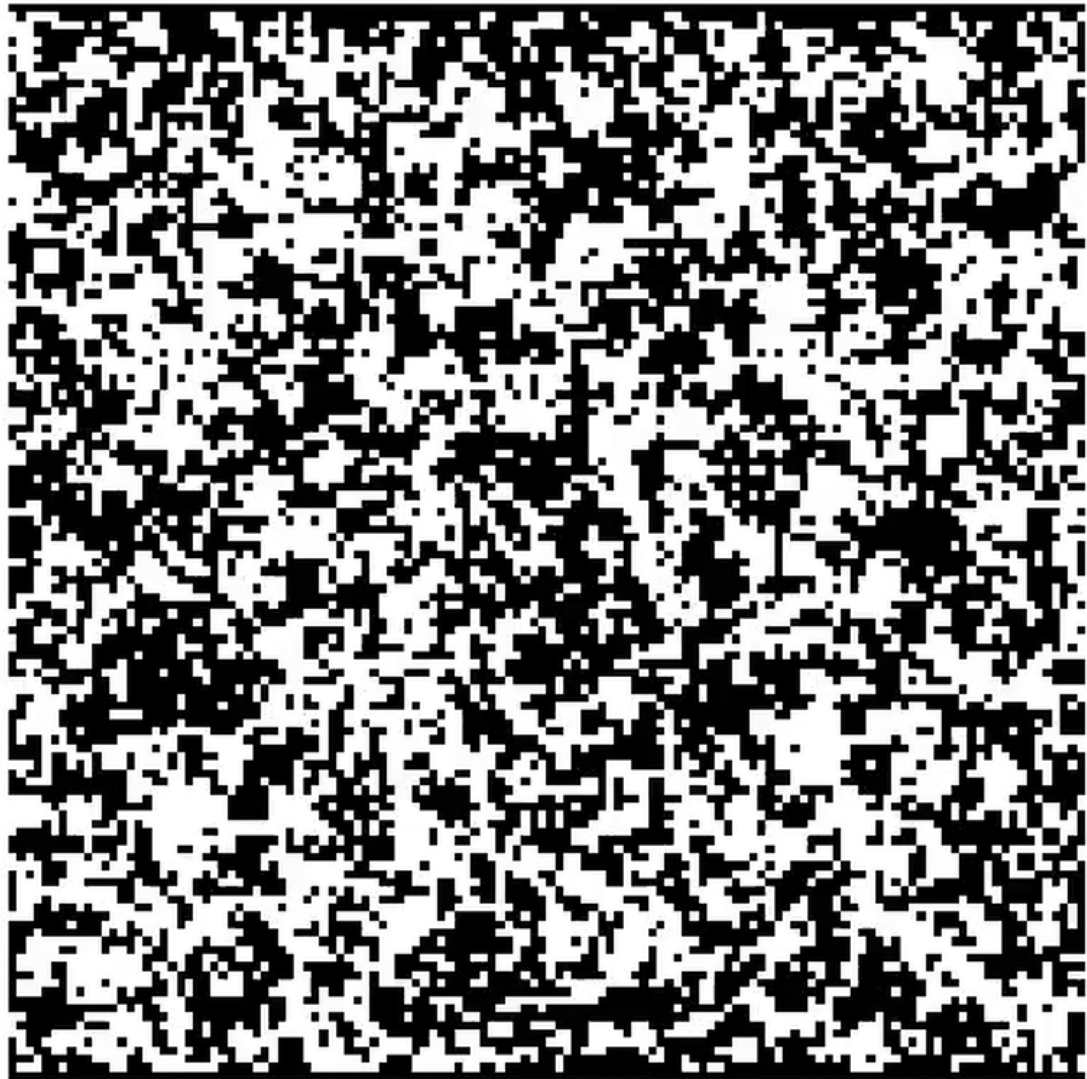


Figura 3: Fotograma de la distribución de espines de la red del modelo de Ising en la dinámica de Kawasaki con magnetización inicial nula y a una temperatura $T = 3.5J/k_B$, con condiciones de contorno periódicas en el eje X y los espines en los bordes superior e inferior de la red fijos.

Como observamos en estas imágenes, a bajas temperaturas, los espines tienden a tomar el mismo valor que sus vecinos, dejando regiones bien diferenciadas de espín $+1$ y -1 . Sin embargo, a grandes temperaturas (por encima de la temperatura crítica), el sistema es mucho más desordenado, mostrando el paramagnetismo del material que se modela. No obstante, aunque es bastante más caótico, sí se pueden observar algunas regiones diferenciadas de distintos espines.

2. $m_0 = 0.50$

Para una magnetización inicial de 0.50 , se presentan los siguientes fotogramas.

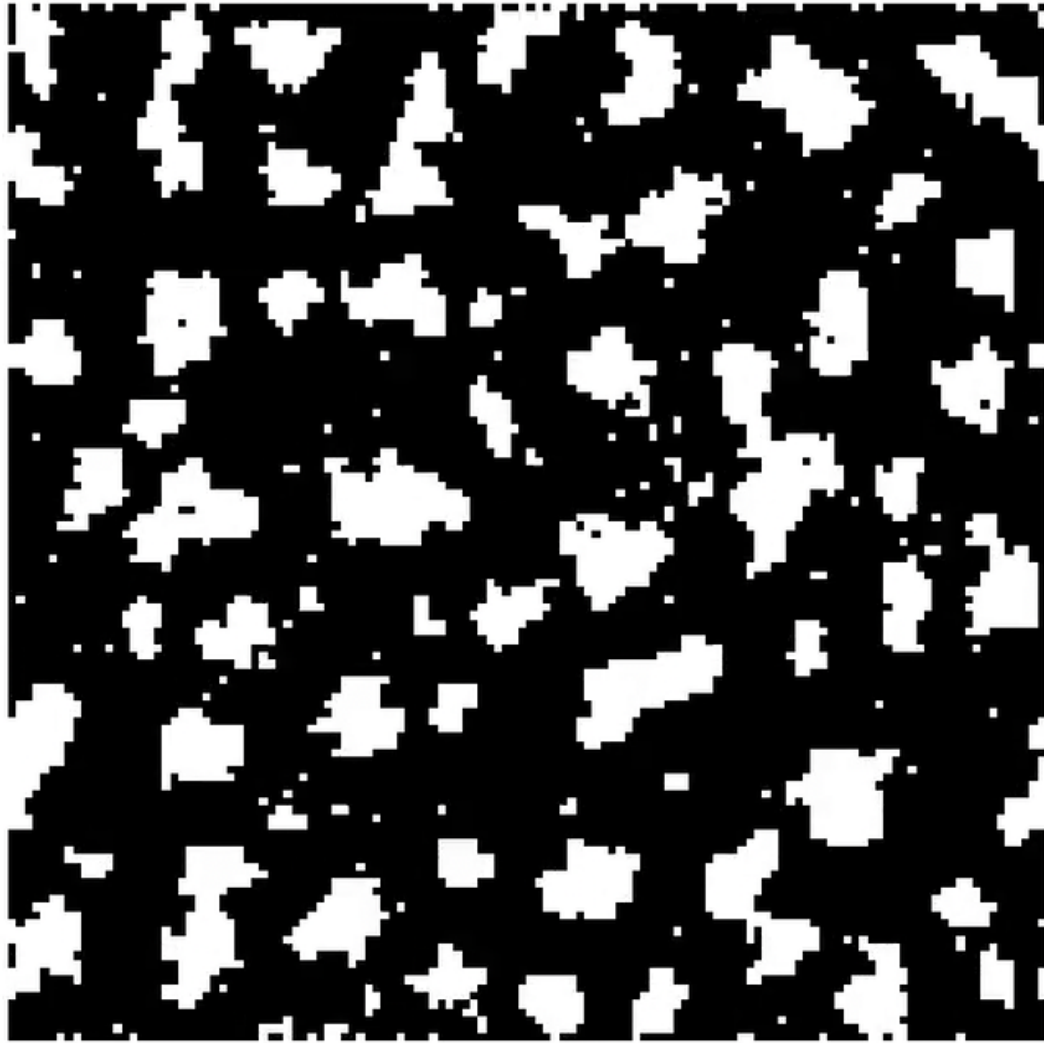


Figura 4: *Fotograma de la distribución de espines de la red del modelo de Ising en la dinámica de Kawasaki con magnetización inicial $m = 0.50$ y a una temperatura $T = 1.50J/k_B$, con condiciones de contorno periódicas en el eje X y los espines en los bordes superior e inferior de la red fijos (aunque no iguales).*

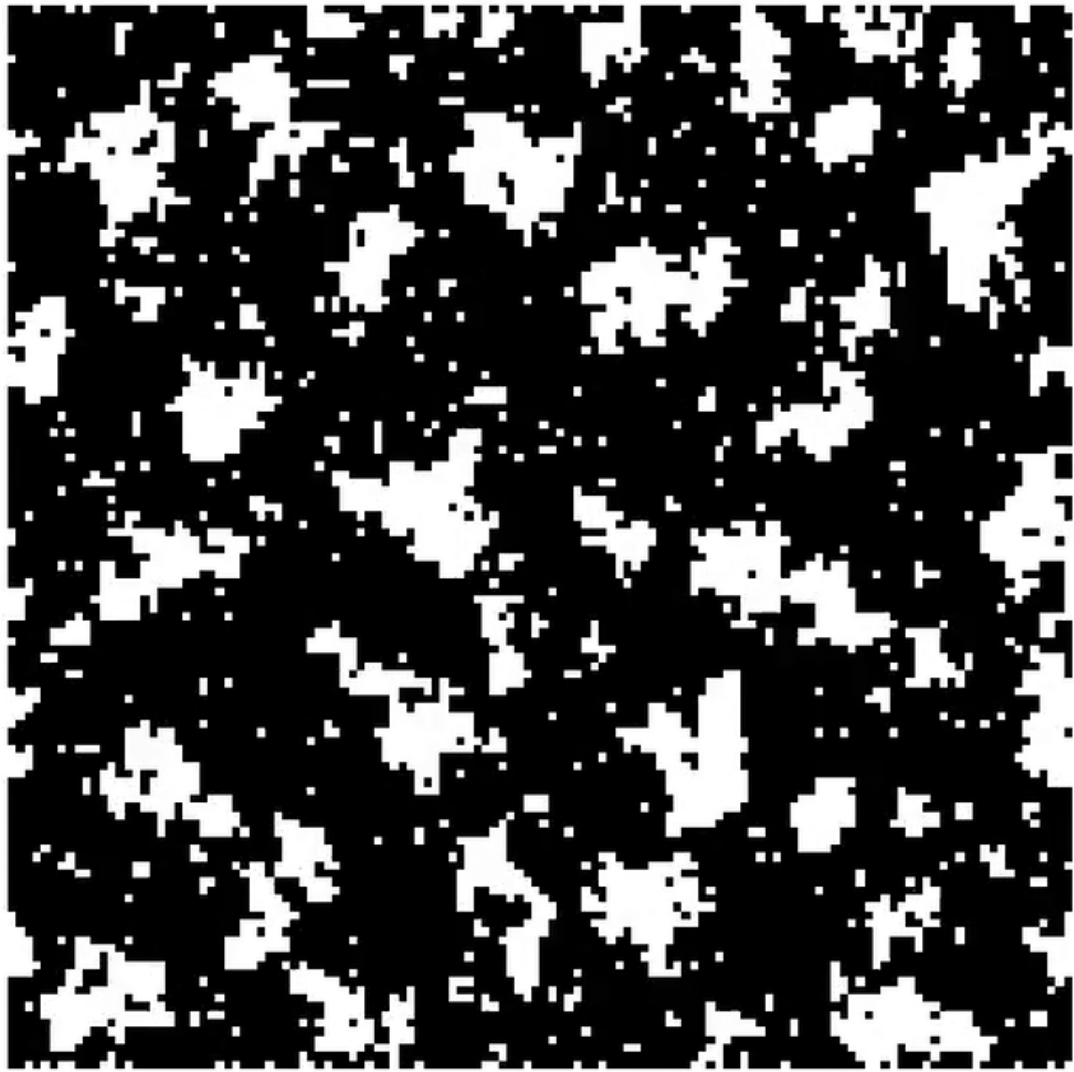


Figura 5: *Fotograma de la distribución de espines de la red del modelo de Ising en la dinámica de Kawasaki con magnetización inicial $m = 0.50$ y a una temperatura $T = 2.10J/k_B$, con condiciones de contorno periódicas en el eje X y los espines en los bordes superior e inferior de la red fijos (aunque no iguales).*

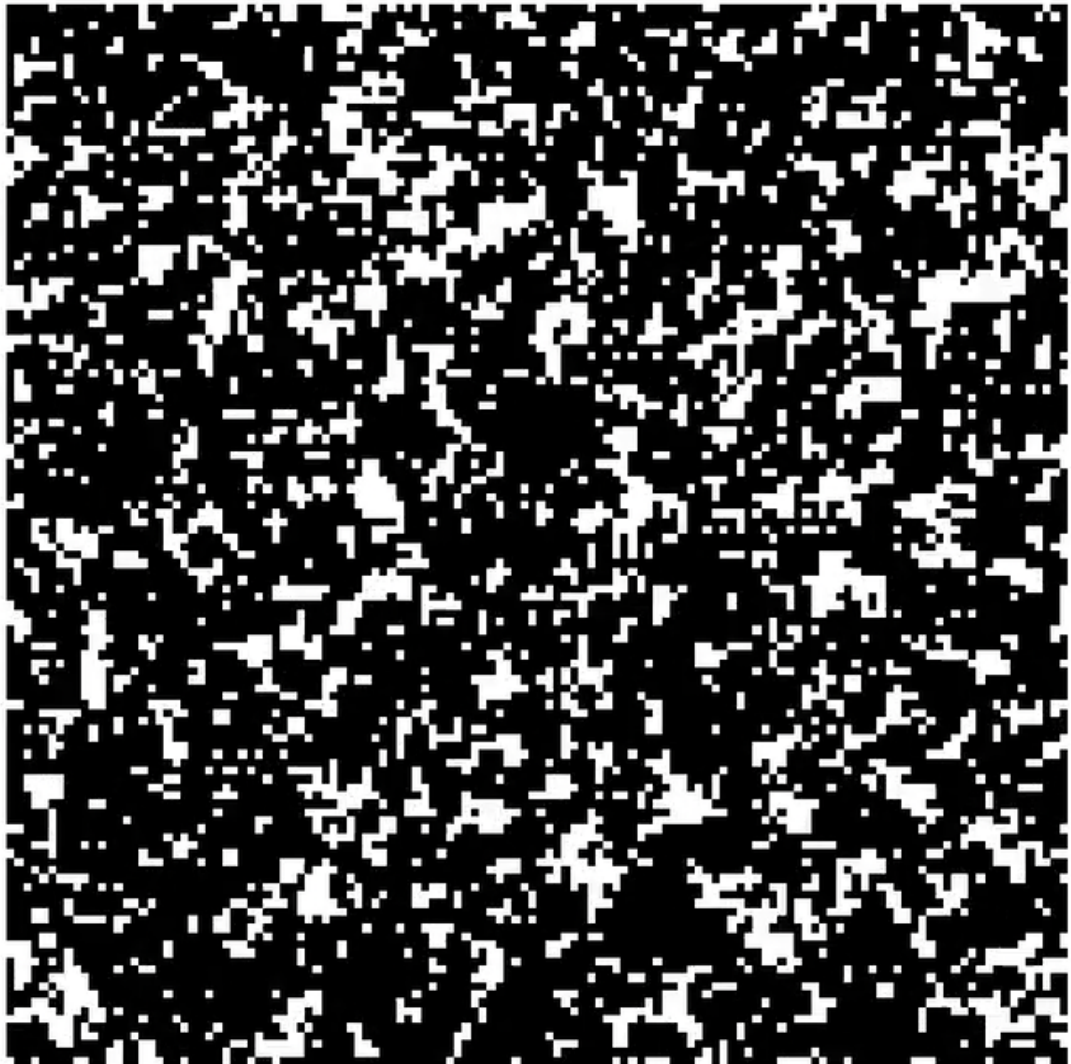


Figura 6: Fotograma de la distribución de espines de la red del modelo de Ising en la dinámica de Kawasaki con magnetización inicial $m = 0.50$ y a una temperatura $T = 3.10J/k_B$, con condiciones de contorno periódicas en el eje X y los espines en los bordes superior e inferior de la red fijos (aunque no iguales).

Con esta magnetización inicial, observamos que la mayoría de espines para alcanzar esa magnetización inicial son -1 . A bajas temperaturas, entre los espines -1 , se crean "gotas" de espín $+1$, diferenciando y ordenando claramente distintas regiones en el sistema. En cuanto va aumentando la temperatura, vemos cómo la red se desordenta hasta quedar un estado muy difuso en el que no hay dominios en los que los espines se encuentren alineados.

3. $m_0 = 0.80$

Para una magnetización inicial de 0.80 , se presentan los siguientes fotogramas.

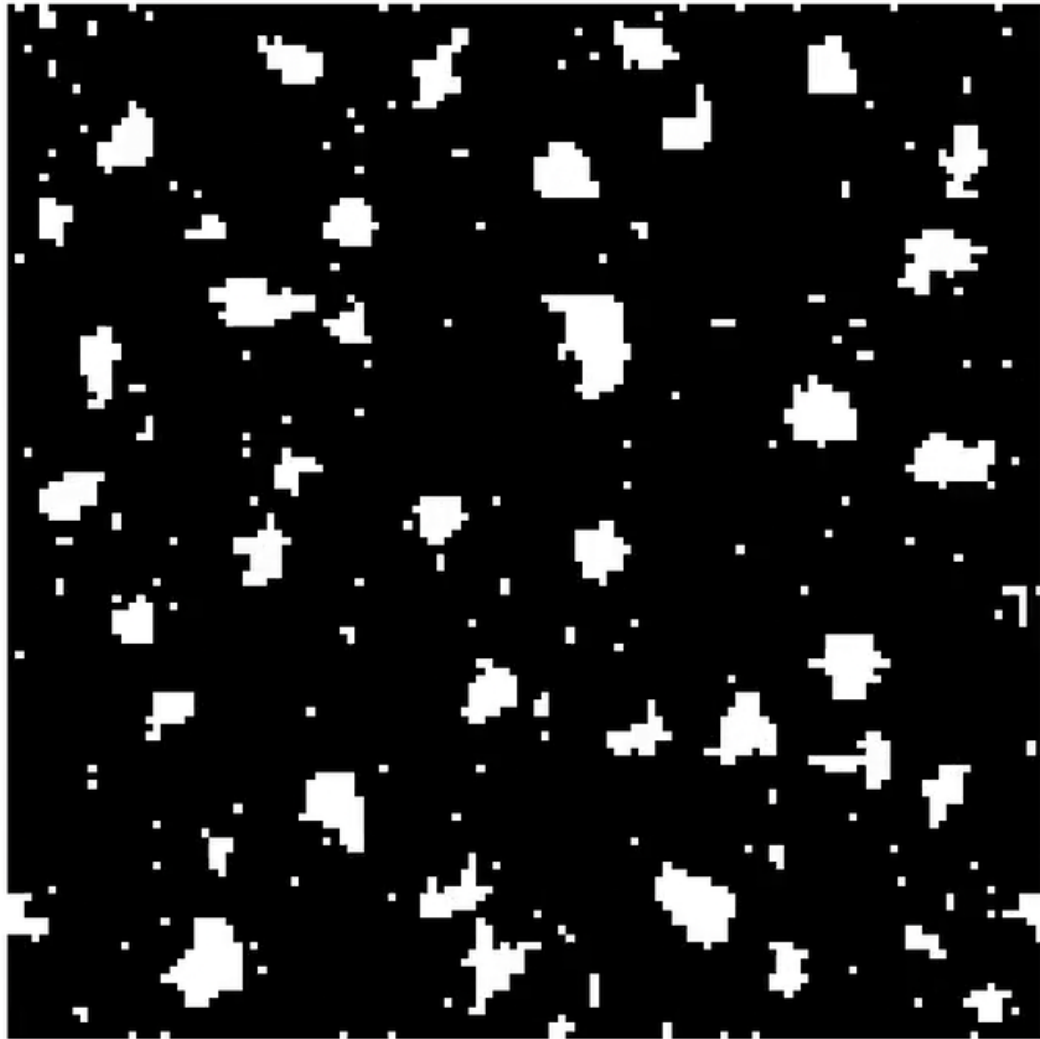


Figura 7: *Fotograma de la distribución de espines de la red del modelo de Ising en la dinámica de Kawasaki con magnetización inicial $m = 0.80$ y a una temperatura $T = 1.50J/k_B$, con condiciones de contorno periódicas en el eje X y los espines en los bordes superior e inferior de la red fijos (aunque no iguales).*

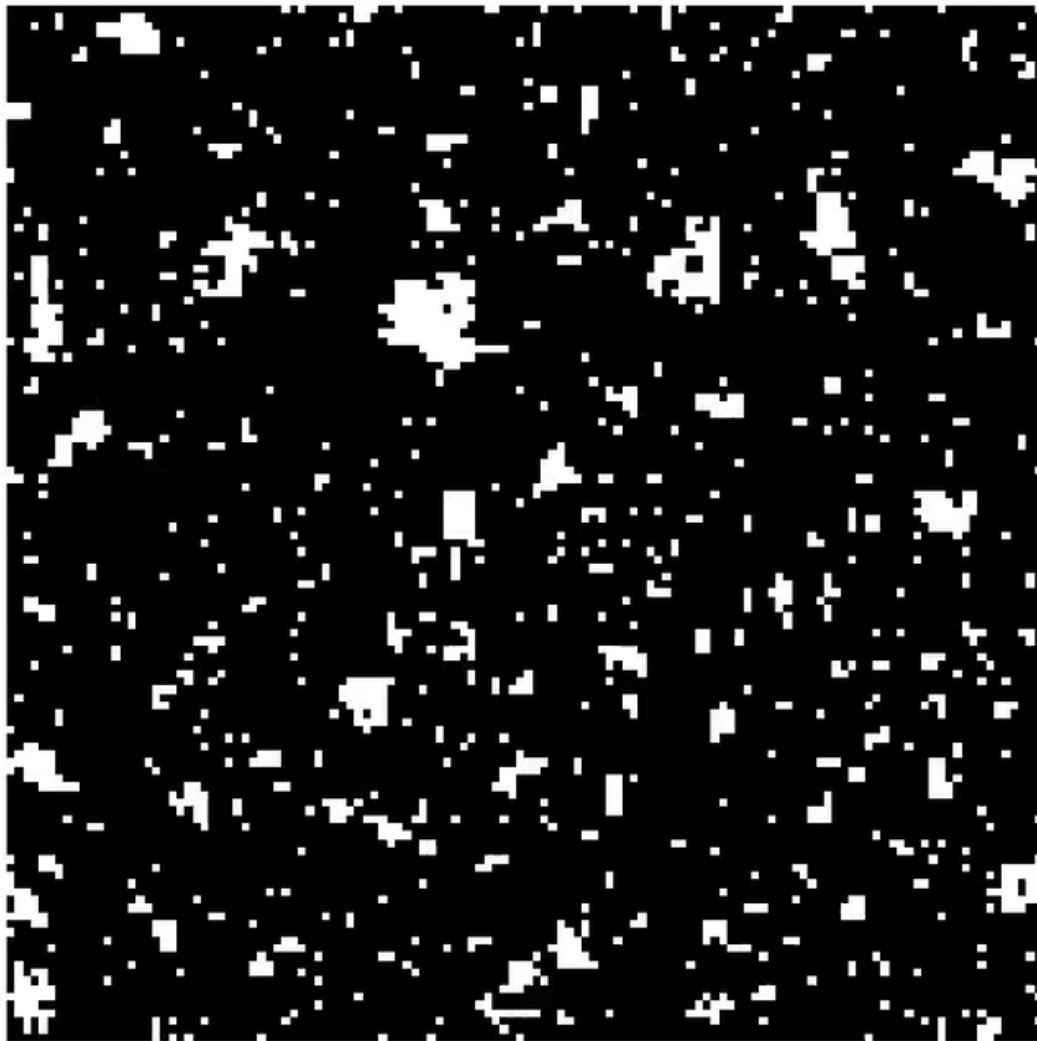


Figura 8: *Fotograma de la distribución de espines de la red del modelo de Ising en la dinámica de Kawasaki con magnetización inicial $m = 0.80$ y a una temperatura $T = 2.10J/k_B$, con condiciones de contorno periódicas en el eje X y los espines en los bordes superior e inferior de la red fijos (aunque no iguales).*

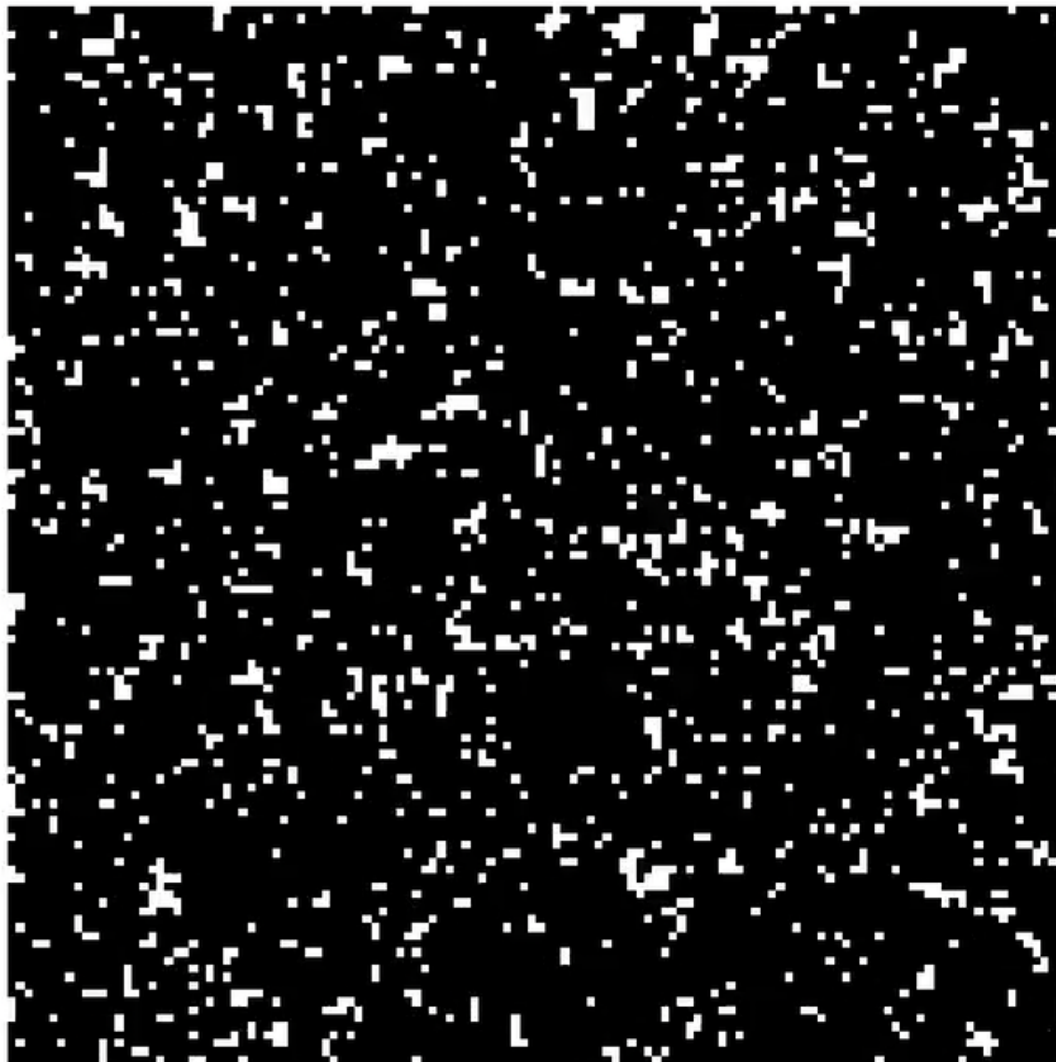


Figura 9: *Fotograma de la distribución de espines de la red del modelo de Ising en la dinámica de Kawasaki con magnetización inicial $m = 0.80$ y a una temperatura $T = 3.10J/k_B$, con condiciones de contorno periódicas en el eje X y los espines en los bordes superior e inferior de la red fijos (aunque no iguales).*

Con esta magnetización inicial, observamos casi todos son -1. En este caso también se crean pequeños dominios de espín +1 entre los demás a temperaturas bajas, e incluso se pueden apreciar algunos a una temperatura intermedia. Sin embargo, a altas temperaturas, los espines +1 se esparcen por la red en muchas más dominios mucho más pequeños.

2. Resultados a magnetización inicial nula.

Tras realizar la simulación, para el programa optimizado, se realizaron unas gráficas que contienen las curvas de magnetización, del calor específico y de la susceptibilidad magnética para cada tamaño de la red de espines.

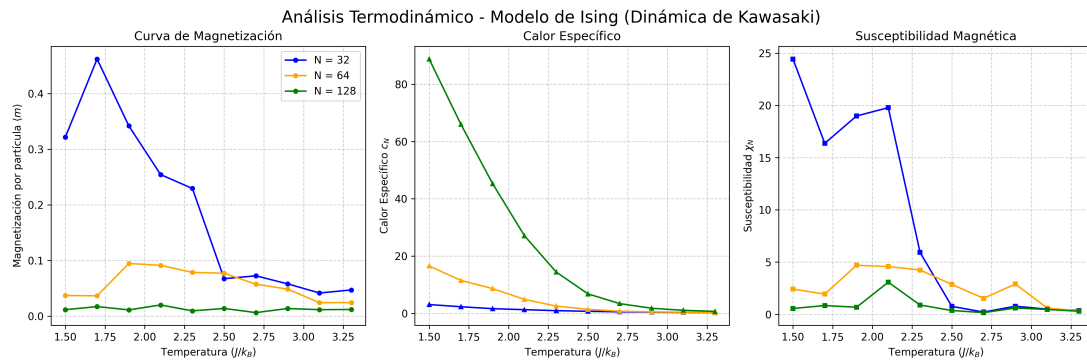


Figura 1: Curvas de magnetización, calor específico y susceptibilidad magnética frente a la temperatura para magnetización inicial nula.

Como vemos, la magnetización del sistema tiende a cero como es de esperar por la dinámica de Kawasaki.

El calor específico calculado, según muestra la gráfica es decreciente en todo momento, teniendo su pico en el valor más pequeño de la temperatura. Esto nos impide vislumbrar en buenas condiciones la temperatura crítica, pues no se corresponde para nada con el valor teórico $2.269 J/k_B$. En vista a los resultados, una extrapolación a un tamaño de red infinito sería a la misma temperatura, pues se podría esperar que el calor específico sea siempre decreciente. Este comportamiento decreciente y anómalo del calor específico a bajas temperaturas es un artificio numérico. Debido a la extrema lentitud de la dinámica de Kawasaki (maduración de Ostwald), la energía en las redes grandes no llega a equilibrarse del todo en el tiempo simulado, inflando artificialmente la varianza de la energía $(\langle E^2 \rangle - \langle E \rangle^2)$ a $T = 1.5 J/k_B$ y enmascarando el verdadero pico crítico.

No obstante, al observar la gráfica de la susceptibilidad magnética sí podemos encontrar la temperatura crítica. Es cierto que para la red de 32×32 espines, encontramos el mismo problema que con el calor específico. Sin embargo, para las redes de 64×64 y 128×128 encontramos unos valores máximos, alcanzados a $1.9 K$ y a $2.1 K$, respectivamente. En vista de los resultados, una extrapolación a un tamaño de red infinita nos daría una temperatura crítica mayor, que convergería al valor teórico $2.269 J/k_B$.

La energía media para distintas temperaturas y distintos tamaños de red se muestra en la siguiente gráfica:

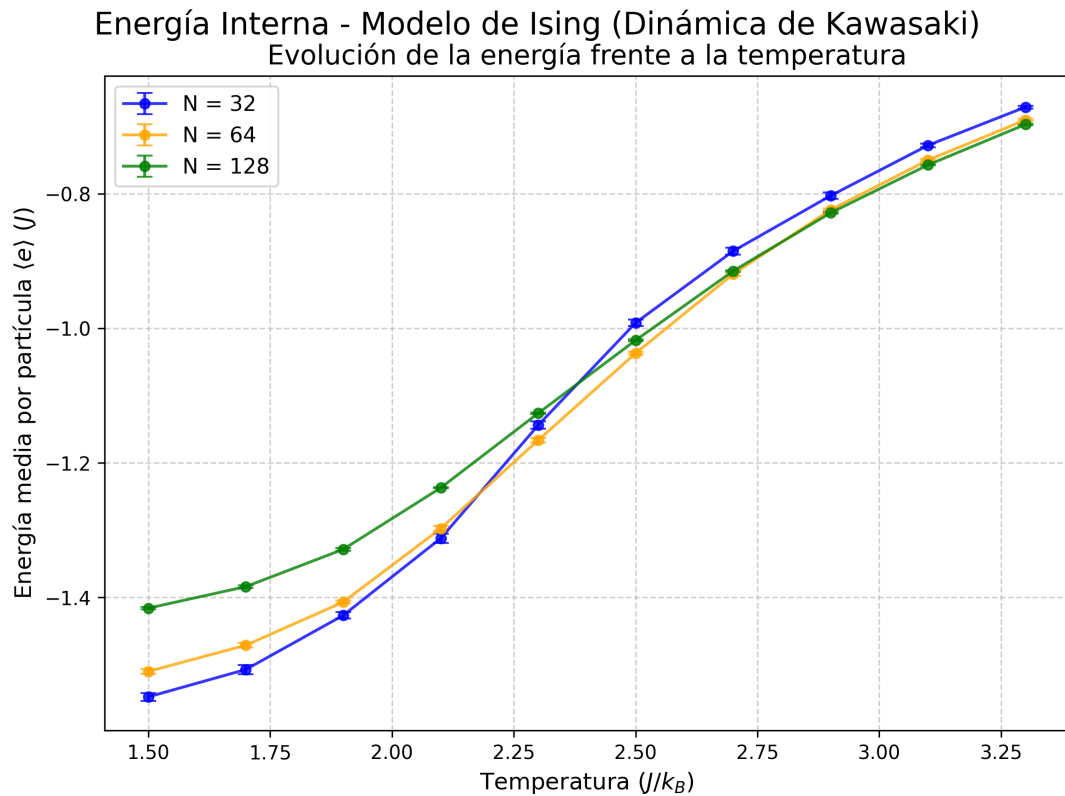


Figura 2: Energía media del sistema a distintas temperaturas y tamaños de red.

3. Densidad de partículas en el eje y en función de temperatura y tamaño.

Al implementar la dinámica de Kawasaki, el número de partículas presentes en el sistema se mantiene estrictamente constante a lo largo del tiempo, independientemente de la temperatura o del estado inicial. Por diseño del algoritmo, las partículas solo permutan sus posiciones espaciales, garantizando la conservación de la masa (o magnetización inicial m_0), lo que resultaría en una evolución temporal de pendiente nula. Esto se debe a que esta dinámica simplemente intercambia espines, lo que impide que se modifique la densidad de partículas.

A continuación se presentan unas gráficas que representa la densidad de partículas promedio en la dirección y para varias temperaturas y posiciones.

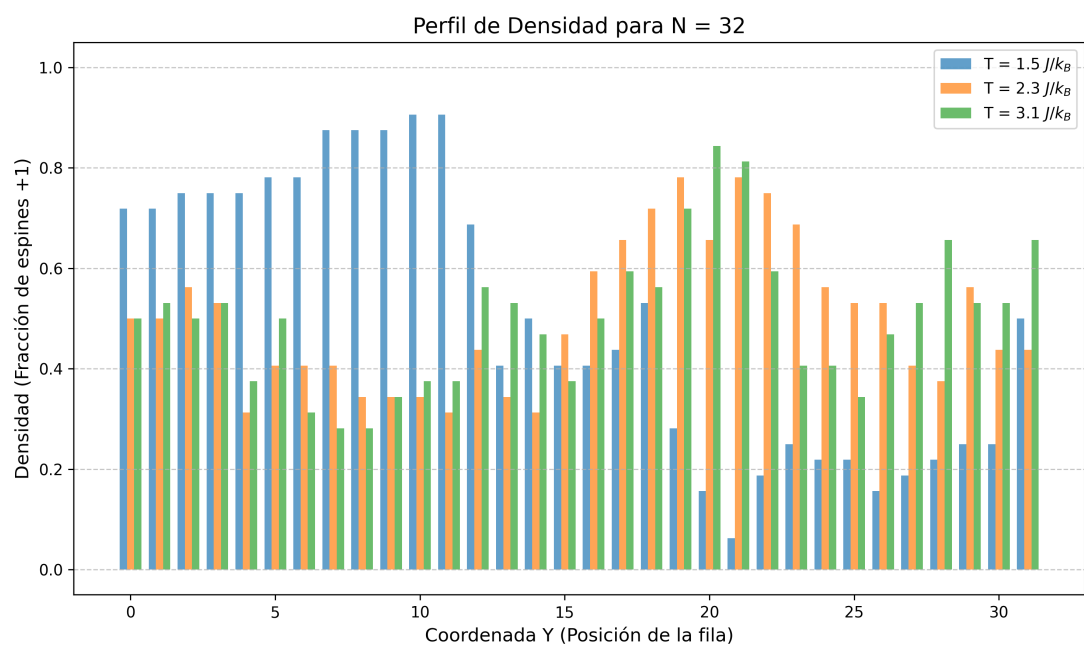


Figura 3: Densidad de partículas promedio en la dirección y para la red de 32×32 y a varias temperaturas.

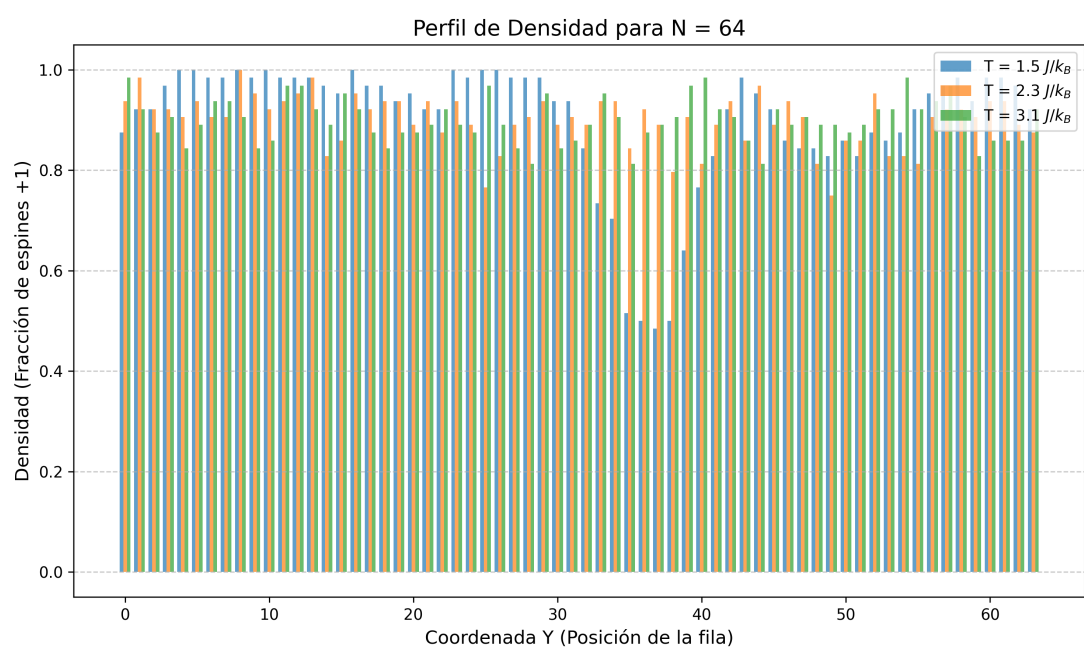


Figura 4: Densidad de partículas promedio en la dirección y para la red de 64×64 y a varias temperaturas.

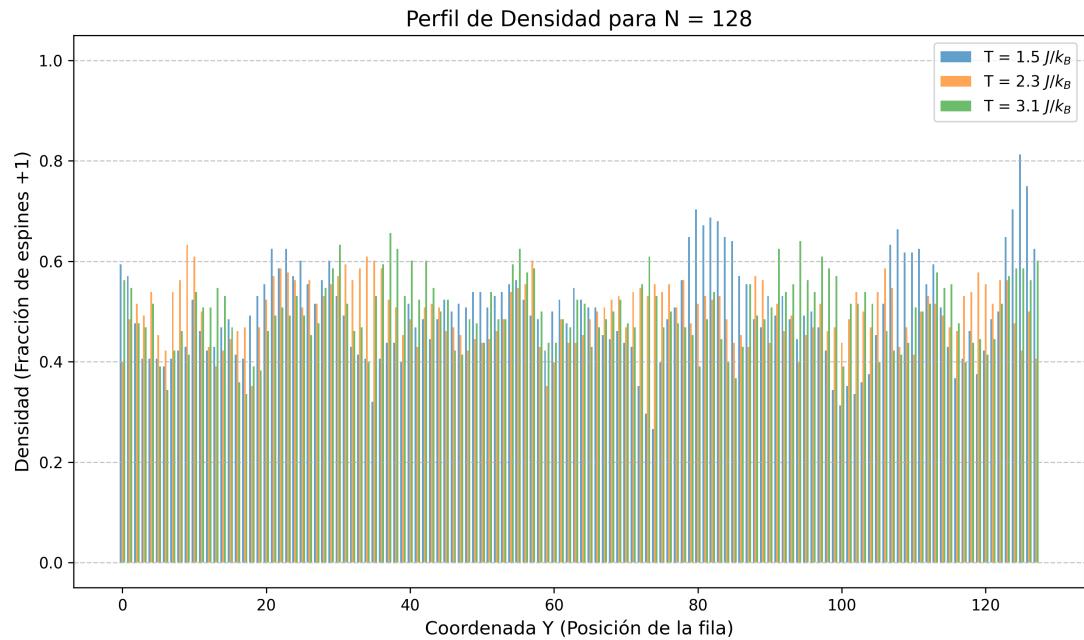


Figura 5: Densidad de partículas promedio en la dirección y para la red de 128x128 y a varias temperaturas.

6. Magnetización inicial no nula.

1. $m_0 = 0.5$

Para una magnetización inicial $m_0 = 0.5$, se obtienen las siguientes curvas de magnetización, calor específico y susceptibilidad magnética.

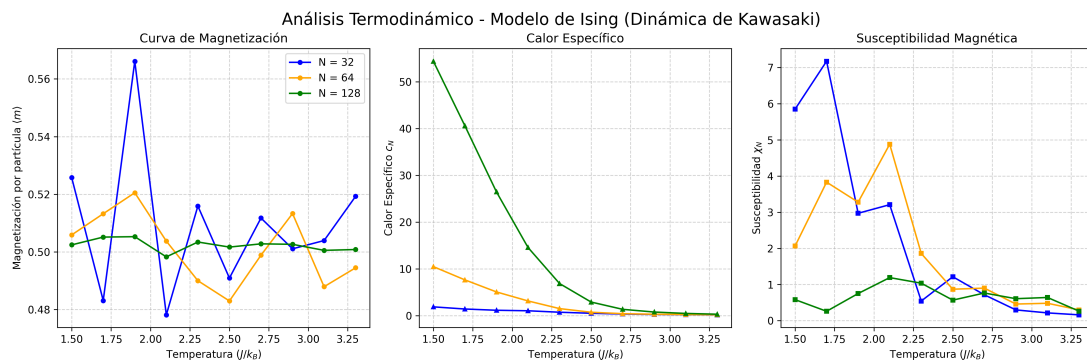


Figura 6: Curvas de magnetización, calor específico y susceptibilidad magnética frente a la temperatura para magnetización inicial de 0.5.

Lo que observamos en este caso es que, salvo unos picos en las redes más pequeñas, la magnetización tiende a mantenerse constante en 0.5. En cuanto a la discontinuidad, no se aprecia bien el salto a cierta temperatura crítica. Sí es cierto que en el salto de $T = 1.90J/k_B$ a $T = 2.10J/k_B$ existe una caída en la magnetización. Lo más probable es que no se aprecie bien esa discontinuidad dada la cantidad limitada de temperaturas para las que se realizaron los experimentos.

En el calor específico volvemos a observar una curva similar a la obtenida en la magnetización nula, siempre decreciente que nos impide conocer la temperatura crítica.

En cuanto a la susceptibilidad magnética, se encuentran picos en $T = 1.70J/k_B$ para la

red de 32×32 y en $T = 2.10J/k_B$ para las redes de 64×64 y 128×128 , siendo estas las temperaturas críticas observadas en este caso. No obstante, cabe destacar que estos picos se hacen más suaves que a temperatura nula. Esto se debe a que el sistema ya no se encuentra en la composición crítica ideal. La transición de fase de segundo orden comienza a emborronarse y la temperatura crítica efectiva a la que se destruyen los dominios mayoritarios se desplaza.

La gráfica de la energía media en función de la temperatura para esta magnetización inicial es la siguiente:

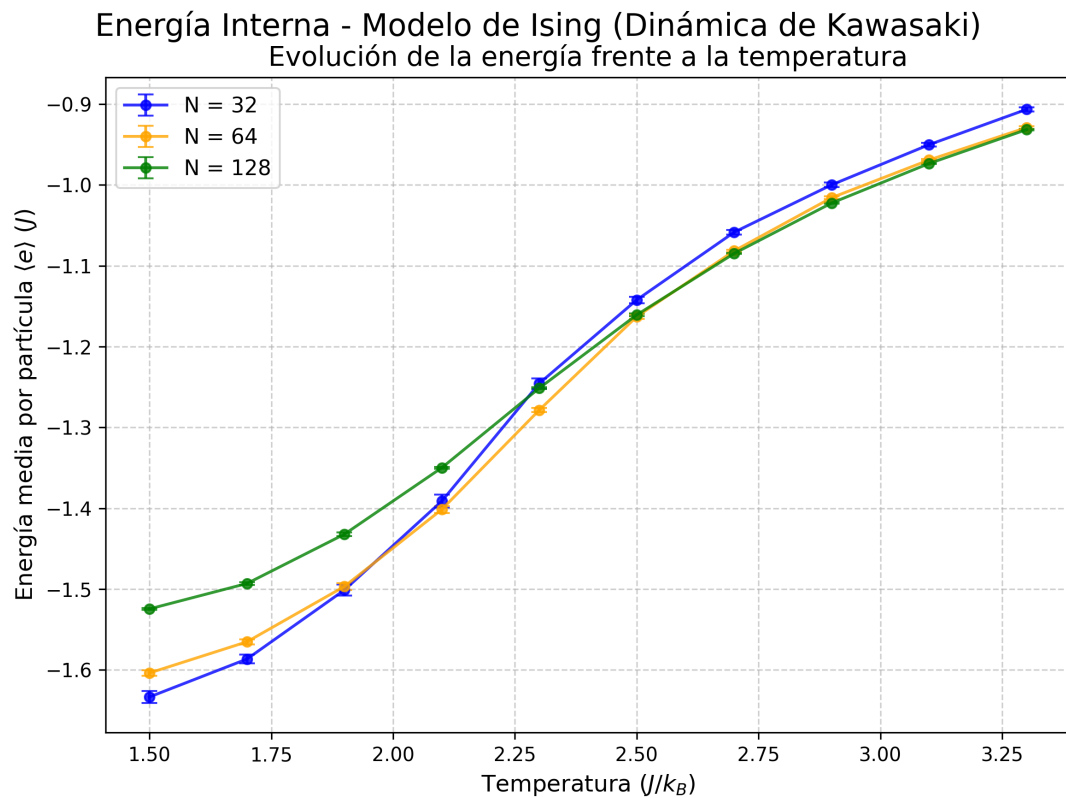


Figura 7: Energía media del sistema a distintas temperaturas y tamaños de red.

A continuación se presentan unas gráficas que representa la densidad de partículas promedio en la dirección y para varias temperaturas.

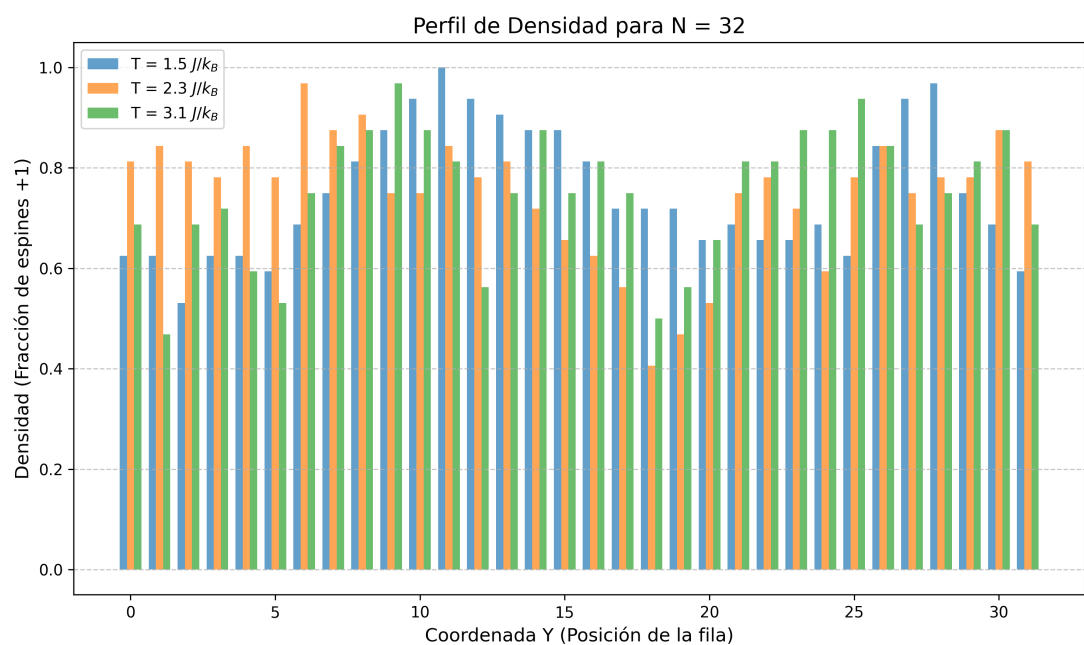


Figura 8: Densidad de partículas promedio en la dirección y para la red de 32×32 y a varias temperaturas.

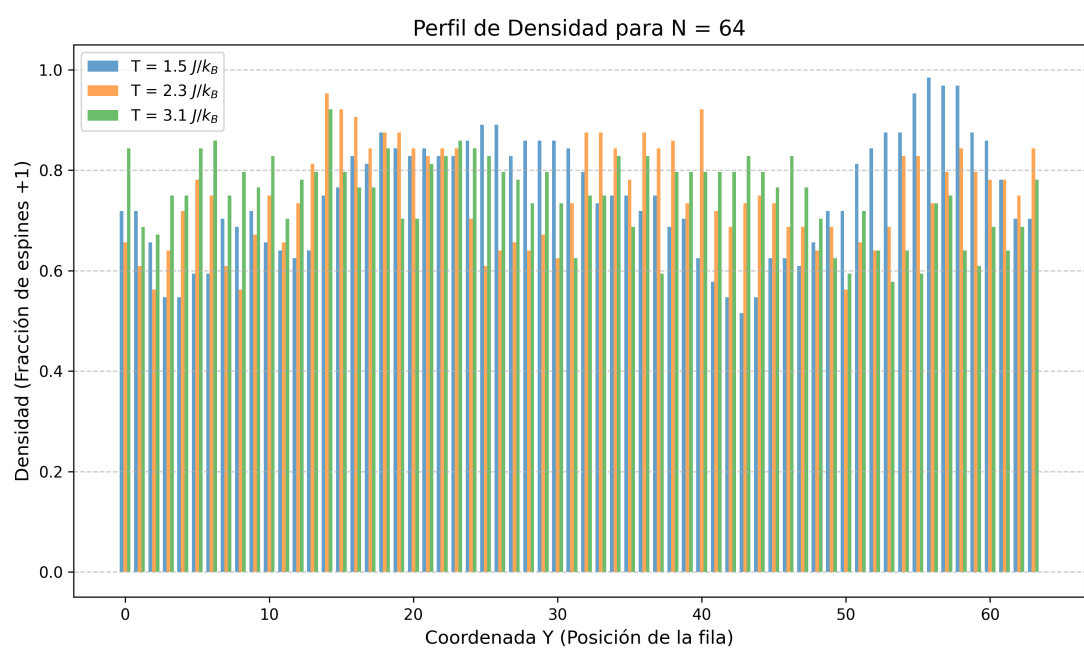


Figura 9: Densidad de partículas promedio en la dirección y para la red de 64×64 y a varias temperaturas.

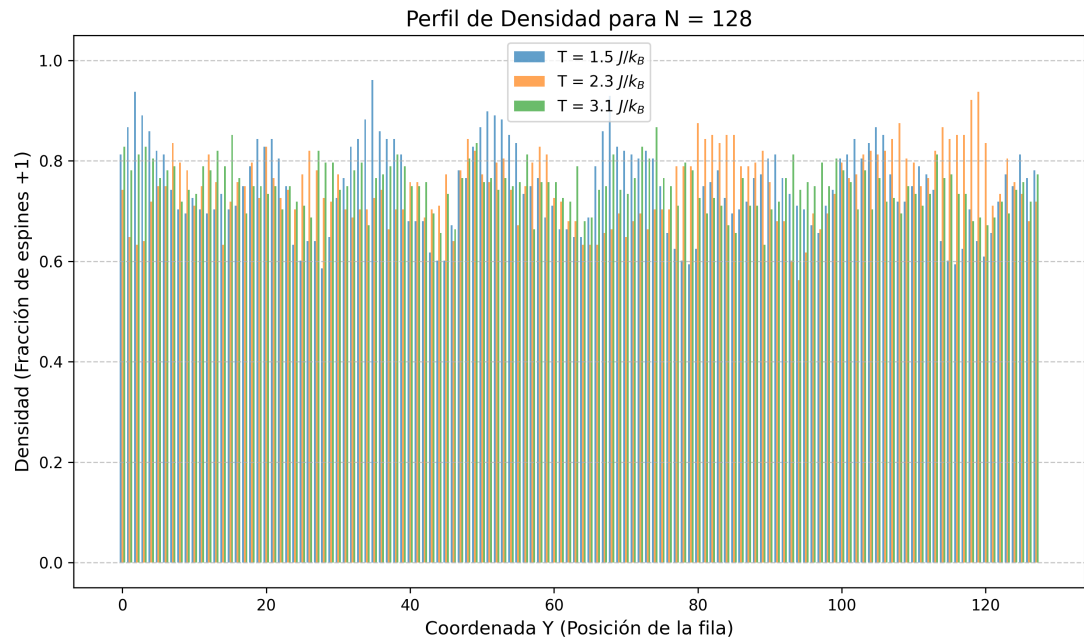


Figura 10: Densidad de partículas promedio en la dirección y para la red de 128×128 y a varias temperaturas.

Nuevamente, dada la naturaleza de Kawasaki, se tiende a mantener constante la densidad de partículas promedio.

2. $m_0 = 0.8$

Para una magnetización inicial $m_0 = 0.8$, se obtienen las siguientes curvas de magnetización, calor específico y susceptibilidad magnética.

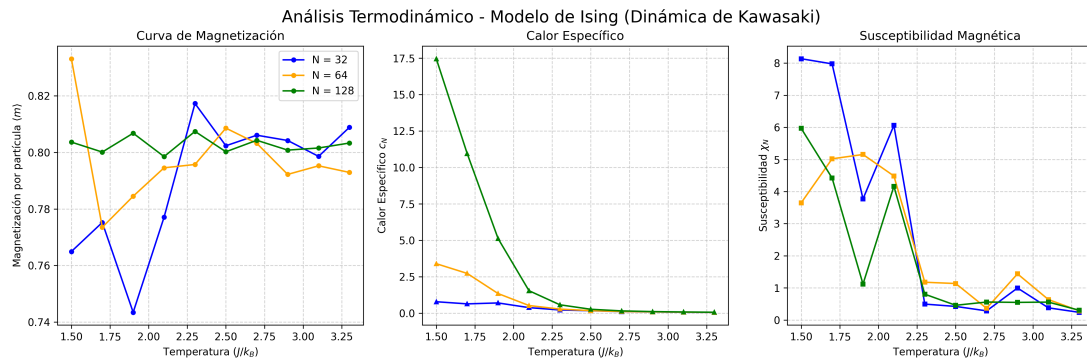


Figura 11: Curvas de magnetización, calor específico y susceptibilidad magnética frente a la temperatura para magnetización inicial de 0.8.

Lo que observamos en este caso es que, en las redes de 32×32 y 64×64 hay unas caídas muy pronunciadas de la magnetización a temperaturas $T = 1.70 J/k_B$ y $T = 1.90 J/k_B$, sin embargo, no se aprecia ninguna discontinuidad muy pronunciada en la curva para 128×128 . Como antes, lo más probable es que se deba al uso de un número escaso de temperaturas, pues al ser una discontinuidad pronunciada, para apreciarla se debería haber tomado valores, por lo menos, muy próximos a la temperatura crítica donde sucede esta discontinuidad. Por otro lado, salvo las discontinuidades, la magnetización tiende a estabilizarse en torno a 0.8.

En el calor específico volvemos a observar una curva similar a la obtenida en la

magnetización nula, siempre decreciente que nos impide conocer la temperatura crítica.

En cuanto a la susceptibilidad magnética, en este caso las redes de 32×32 y 128×128 encuentran su pico a la temperatura inicial $1.50 J/k_B$, mientras que la de la red de 64×64 tiene su máximo en $1.90 J/k_B$. Con estos resultados no puedo llegar a ninguna conclusión clara sobre la temperatura crítica. Además, con mayor claridad que en el caso $m_0 = 0.50$, se puede observar que curva se suaviza más en torno a los máximos.

La gráfica de la energía media en función de la temperatura para esta magnetización inicial es la siguiente:



Figura 12: Energía media del sistema a distintas temperaturas y tamaños de red.

A continuación se presentan unas gráficas que representa la densidad de partículas promedio en la dirección y para varias temperaturas.

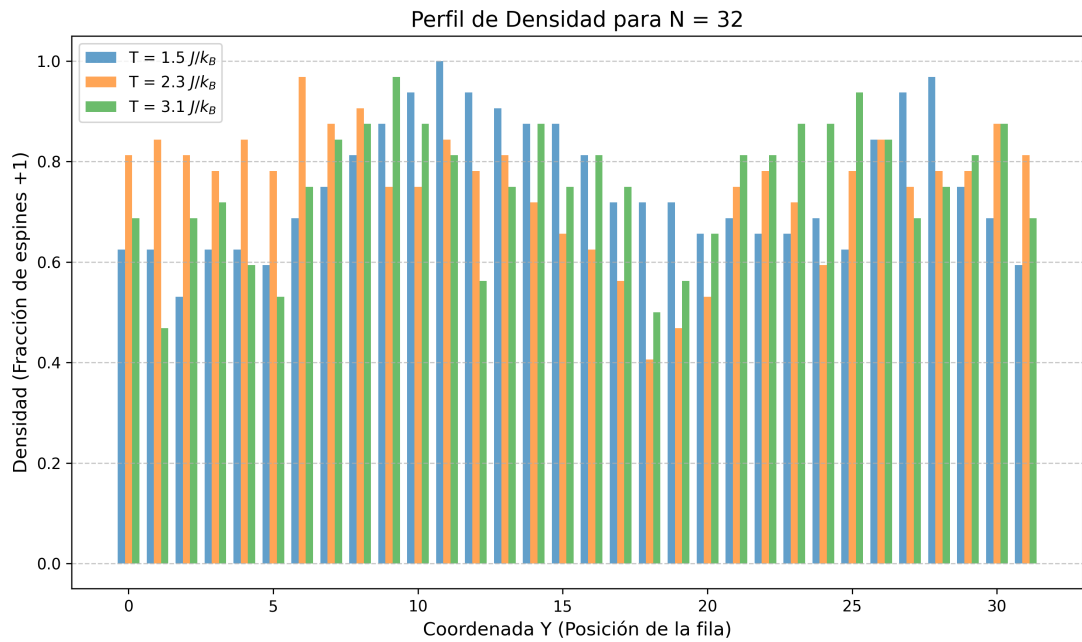


Figura 13: Densidad de partículas promedio en la dirección y para la red de 32×32 y a varias temperaturas.

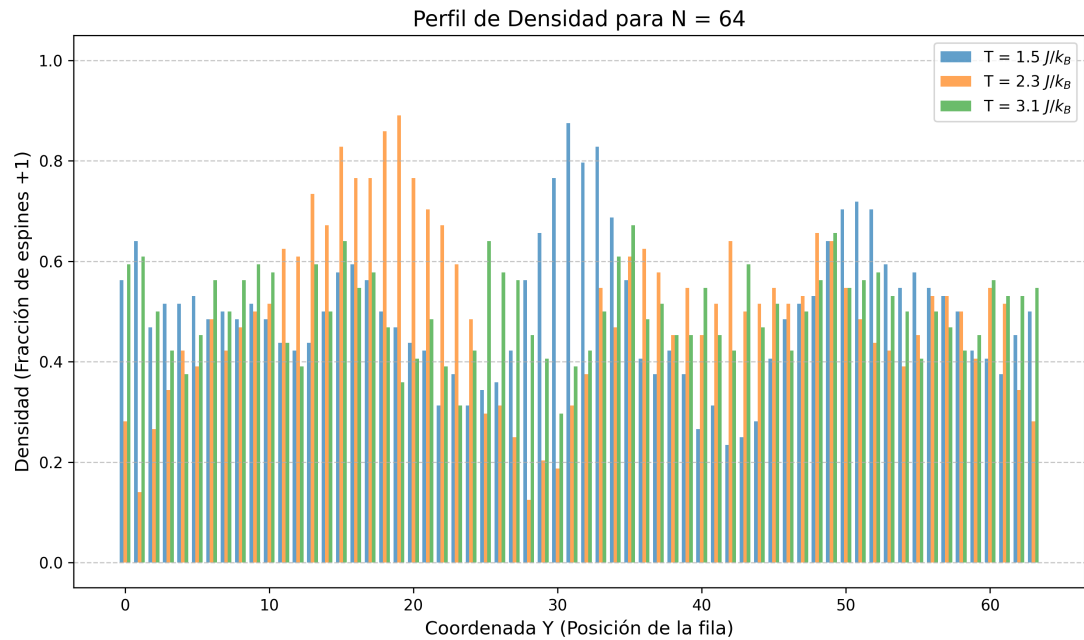


Figura 14: Densidad de partículas promedio en la dirección y para la red de 64x64 y a varias temperaturas.

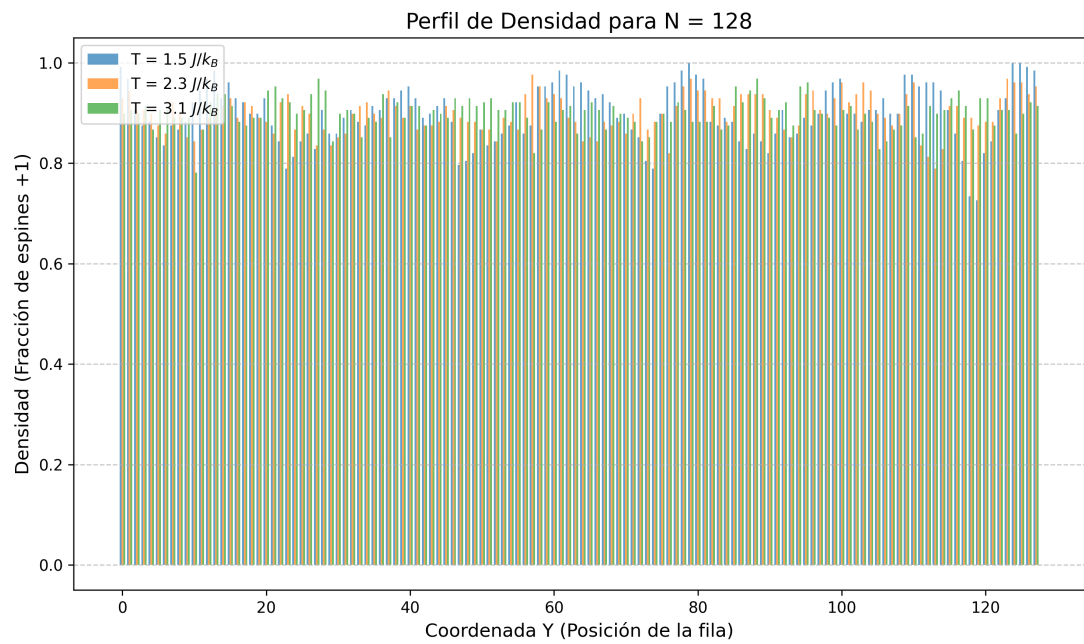


Figura 15: Densidad de partículas promedio en la dirección y para la red de 128x128 y a varias temperaturas.

Nuevamente, dada la naturaleza de Kawasaki, se tiende a mantener constante la densidad de partículas promedio.

7. Comparaciones.

Al realizar la simulación en distintas condiciones, obtenemos resultados suficientemente similares, por lo que solo se ha analizado en detalle una de las simulaciones. La comparativa de eficiencia entre el programa optimizado por OpenMP y el no optimizado es notoria, pues el programa no optimizado tardó en ejecutarse unas 4 horas, mientras que el paralelizado redujo el tiempo de ejecución a algo menos de 2 horas.

Para comparar con el cluster del departamento de física estadística de la universidad de granada (joel), en busca de economizar el tiempo, tan solo se ejecutó el programa optimizado. En joel, el programa tardó en ejecutarse:

5. Conclusiones.

Como conclusión, en esta práctica hemos podido observar claramente cómo las condiciones de temperatura de un sistema afectan a su magnetización. Hemos observado gracias a las gráficas de la red de espines como a bajas temperaturas se crean dominios bien diferenciados de espín $+1$ y -1 , tal y como sucede en materiales ferromagnéticos. A diferentes magnetizaciones iniciales, se ha observado que cuanto mayor es su valor, mayor es la densidad de espines -1 . Asimismo, en casos de magnetizaciones iniciales no nulas, también se observa este mismo fenómeno.

A través de las curvas de magnetización y de susceptibilidad magnética, se ha podido vislumbrar la existencia de una temperatura crítica donde las características magnéticas del sistema pasan del ferromagnetismo al paramagnetismo. También se ha podido apreciar una mejora notoria en los resultados obtenidos a mayor tamaño de la red de espines, lo que nos lleva a la conclusión de que a un tamaño infinito nuestras aproximaciones son correctas. Sin embargo, no se han podido observar resultados referentes a esta temperatura con las gráficas del calor específico.

Por último, se ha comprobado que la dinámica de Kawasaki, como era de esperar, conserva la densidad total de partículas de cada espín, pues solo se intercambian posiciones (a diferencia de la dinámica de Glauber).

Bibliografía

Voluntario 2: Simulación del modelo de Ising con la dinámica de Kawasaki - Profesorado de Física Computacional de la UGR. *Modelo de Ising: Una Introducción al Método Monte Carlo (diapositivas de clase)* - Profesorado de Física Computacional de la UGR. *(Breve) Cálculo de errores y Técnicas Montecarlo* - Daniel Manzano.